

目录

java 面试笔试题大汇总-----	1
面试时候经常会问的一些问题-----	27
神州数码面试集合-----	61
长城软件笔试-----	69
EDS 的面试题-----	71
盛意科技 java 面试-----	72

Tarena Wuhan

java 面试笔试题大汇总

分类:J2EE 学习

java 面试笔试题大汇总

JAVA 相关基础知识

1、面向对象的特征有哪些方面

1.抽象:

抽象就是忽略一个主题中与当前目标无关的那些方面,以便更充分地注意与当前目标有关的方面。抽象并不打算了解全部问题,而只是选择其中的一部分,暂时不用部分细节。抽象包括两个方面,一是过程抽象,二是数据抽象。

2.继承:

继承是一种联结类的层次模型,并且允许和鼓励类的重用,它提供了一种明确表述共性的方法。对象的一个新类可以从现有的类中派生,这个过程称为类继承。新类继承了原始类的特性,新类称为原始类的派生类(子类),而原始类称为新类的基类(父类)。派生类可以从它的基类那里继承方法和实例变量,并且类可以修改或增加新的方法使之更适合特殊的需要。

3.封装:

封装是把过程和数据包围起来,对数据的访问只能通过已定义的界面。面向对象计算始于这个基本概念,即现实世界可以被描绘成一系列完全自治、封装的对象,这些对象通过一个受保护的接口访问其他对象。

4. 多态性:

多态性是指允许不同的对象对同一消息作出响应。多态性包括参数化多态性和包含多态性。多态性语言具有灵活、抽象、行为共享、代码共享的优势,很好的解决了应用程序函数同名问题。

2、String 是最基本的数据类型吗?

基本数据类型包括 byte、int、char、long、float、double、boolean 和 short。

java.lang.String 类是 final 类型的,因此不可以继承这个类、不能修改这个类。为了提高效率节省空间,我们应该用 StringBuffer 类

3、int 和 Integer 有什么区别

Java 提供两种不同的类型: 引用类型和原始类型(或内置类型)。Int 是 java 的原始数据类型, Integer 是 java 为 int 提供的封装类。Java 为每个原始类型提供了封装类。

原始类型封装类

booleanBoolean

charCharacter

byteByte

shortShort

intInteger

longLong

floatFloat

doubleDouble

引用类型和原始类型的行为完全不同，并且它们具有不同的语义。引用类型和原始类型具有不同的特征和用法，它们包括：大小和速度问题，这种类型以哪种类型的数据结构存储，当引用类型和原始类型用作某个类的实例数据时所指定的缺省值。对象引用实例变量的缺省值为 `null`，而原始类型实例变量的缺省值与它们的类型有关。

4、String 和 StringBuffer 的区别

JAVA 平台提供了两个类：`String` 和 `StringBuffer`，它们可以储存和操作字符串，即包含多个字符的字符数据。这个 `String` 类提供了数值不可改变的字符串。而这个 `StringBuffer` 类提供的字符串进行修改。当你知道字符数据要改变的时候你就可以使用 `StringBuffer`。典型地，你可以使用 `StringBuffers` 来动态构造字符数据。

5、运行时异常与一般异常有何异同？

异常表示程序运行过程中可能出现的非正常状态，运行时异常表示虚拟机的通常操作中可能遇到的异常，是一种常见运行错误。`java` 编译器要求方法必须声明抛出可能发生的非运行时异常，但是并不要求必须声明抛出未被捕获的运行时异常。

6、说出 Servlet 的生命周期，并说出 Servlet 和 CGI 的区别。

`Servlet` 被服务器实例化后，容器运行其 `init` 方法，请求到达时运行其 `service` 方法，`service` 方法自动派遣运行与请求对应的 `doXXX` 方法（`doGet`，`doPost`）等，当服务器决定将实例销毁的时候调用其 `destroy` 方法。

与 `cgi` 的区别在于 `servlet` 处于服务器进程中，它通过多线程方式运行其 `service` 方法，一个实例可以服务于多个请求，并且其实例一般不会销毁，而 `CGI` 对每个请求都产生新的进程，服务完成后就销毁，所以效率上低于 `servlet`。

7、说出 ArrayList, Vector, LinkedList 的存储性能和特性

`ArrayList` 和 `Vector` 都是使用数组方式存储数据，由于数组元素数大于实际存储的数据以便增加和插入元素，它们都允许直接按序号索引元素，但是插入元素要涉及数组元素移动等内存操作，所以索引数据快而插入数据慢。`Vector` 由于使用了 `synchronized` 方法（线程安全），通常性能上较 `ArrayList` 差，而 `LinkedList` 使用双向链表实现存储，按序号索引数据需要进行前向或后向遍历，但是插入数据时只需要记录本项的前后项即可，所以插入速度较快。

8、EJB 是基于哪些技术实现的？并说出 SessionBean 和 EntityBean 的区别，StatefulBean 和 StatelessBean 的区别。

EJB 包括 `Session Bean`、`Entity Bean`、`Message Driven Bean`，基于 `JNDI`、`RMI`、`JAT` 等技术实现。

`SessionBean` 在 `J2EE` 应用程序中被用来完成一些服务器端的业务操作，例如访问数据库、调用其他 EJB 组件。`EntityBean` 被用来代表应用系统中用到的数据。

对于客户机，`SessionBean` 是一种非持久性对象，它实现某些在服务器上运行的业务逻辑。

对于客户机，`EntityBean` 是一种持久性对象，它代表一个存储在持久性存储器中的实体的对象视图，或是一个由现有企业应用程序实现的实体。

`Session Bean` 还可以再细分为 `Stateful Session Bean` 与 `Stateless Session Bean`，这两种的 `Session Bean` 都可以将系统逻辑放在 `method` 之中执行，不同的是 `Stateful Session Bean` 可以记录呼叫者的状态，因此通常来说，一个使用者会有一个相对应的 `Stateful Session Bean` 的实体。`Stateless Session Bean` 虽然也是逻辑组件，但是他却不负责记录使用者状态，也就是说当使用者呼叫 `Stateless Session Bean` 的时候，EJB Container 并不会找寻特定的 `Stateless Session Bean` 的实体来执行这个 `method`。换言之，很可能数个使用者在执行某个 `Stateless Session Bean` 的 `methods` 时，会是同一个 `Bean` 的 `Instance` 在执行。从内存方面来看，`Stateful Session Bean` 与 `Stateless Session Bean` 比较，`Stateful Session Bean` 会消耗

J2EE Server 较多的内存，然而 Stateful Session Bean 的优势却在于他可以维持使用者的状态。

9、Collection 和 Collections 的区别。

Collection 是集合类的上级接口，继承与他的接口主要有 Set 和 List。

Collections 是针对集合类的一个帮助类，他提供一系列静态方法实现对各种集合的搜索、排序、线程安全化等操作。

10、&和&&的区别。

&是位运算符，表示按位与运算，&&是逻辑运算符，表示逻辑与（and）。

11、HashMap 和 Hashtable 的区别。

HashMap 是 Hashtable 的轻量级实现（非线程安全的实现），他们都完成了 Map 接口，主要区别在于 HashMap 允许空(null)键值(key),由于非线程安全,效率上可能高于 Hashtable。HashMap 允许将 null 作为一个 entry 的 key 或者 value，而 Hashtable 不允许。HashMap 把 Hashtable 的 contains 方法去掉了，改成 containsvalue 和 containsKey。因为 contains 方法容易让人引起误解。

Hashtable 继承自 Dictionary 类，而 HashMap 是 Java1.2 引进的 Map interface 的一个实现。最大的不同是，Hashtable 的方法是 Synchronize 的，而 HashMap 不是，在多个线程访问 Hashtable 时，不需要自己为它的方法实现同步，而 HashMap 就必须为之提供外同步。Hashtable 和 HashMap 采用的 hash/rehash 算法都大概一样，所以性能不会有很大的差异。

12、final, finally, finalize 的区别。

final 用于声明属性，方法和类，分别表示属性不可变，方法不可覆盖，类不可继承。finally 是异常处理语句结构的一部分，表示总是执行。finalize 是 Object 类的一个方法，在垃圾收集器执行的时候会调用被回收对象的此方法，可以覆盖此方法提供垃圾收集时的其他资源回收，例如关闭文件等。

13、sleep() 和 wait() 有什么区别？

sleep 是线程类（Thread）的方法，导致此线程暂停执行指定时间，给执行机会给其他线程，但是监控状态依然保持，到时后会自动恢复。调用 sleep 不会释放对象锁。wait 是 Object 类的方法，对此对象调用 wait 方法导致本线程放弃对象锁，进入等待此对象的等待锁定池，只有针对此对象发出 notify 方法（或 notifyAll）后本线程才进入对象锁定池准备获得对象锁进入运行状态。

14、Overload 和 Override 的区别。Overloaded 的方法是否可以改变返回值的类型？

方法的重写 Overriding 和重载 Overloading 是 Java 多态性的不同表现。重写 Overriding 是父类与子类之间多态性的一种表现，重载 Overloading 是一个类中多态性的一种表现。如果在子类中定义某方法与其父类有相同的名称和参数，我们说该方法被重写 (Overriding)。子类的对象使用这个方法时，将调用子类中的定义，对它而言，父类中的定义如同被“屏蔽”了。如果在一个类中定义了多个同名的方法，它们或有不同的参数个数或有不同的参数类型，则称为方法的重载(Overloading)。Overloaded 的方法是可以改变返回值的类型。

15、error 和 exception 有什么区别？

error 表示恢复不是不可能但很困难的情况下的一种严重问题。比如说内存溢出。不可能指望程序能处理这样的情况。

exception 表示一种设计或实现问题。也就是说，它表示如果程序运行正常，从不会发生的情况。

16、同步和异步有何异同，在什么情况下分别使用他们？举例说明。

如果数据将在线程间共享。例如正在写的的数据以后可能被另一个线程读到，或者正在读的数据可能已经被另一个线程写过了，那么这些数据就是共享数据，必须进行同步存取。

当应用程序在对象上调用了—个需要花费很长时间来执行的方法,并且不希望让程序等待方法的返回时,就应该使用异步编程,在很多情况下采用异步途径往往更有效率。

17、abstract class 和 interface 有什么区别?

声明方法的存在而不去实现它的类被叫做抽象类 (abstract class), 它用于要创建一个体现某些基本行为的类, 并为该类声明方法, 但不能在该类中实现该方法的情况。不能创建 abstract 类的实例。然而可以创建一个变量, 其类型是一个抽象类, 并让它指向具体子类的一个实例。不能有抽象构造函数或抽象静态方法。Abstract 类的子类为它们父类中的所有抽象方法提供实现, 否则它们也是抽象类。取而代之, 在子类中实现该方法。知道其行为的其它类可以在类中实现这些方法。

接口 (interface) 是抽象类的变体。在接口中, 所有方法都是抽象的。多继承性可通过实现这样的接口而获得。接口中的所有方法都是抽象的, 没有一个有程序体。接口只可以定义 static final 成员变量。接口的实现与子类相似, 除了该实现类不能从接口定义中继承行为。当类实现特殊接口时, 它定义 (即将程序体给予) 所有这种接口的方法。然后, 它可以在实现了该接口的类的任何对象上调用接口的方法。由于有抽象类, 它允许使用接口名作为引用变量的类型。通常的动态联编将生效。引用可以转换到接口类型或从接口类型转换, instanceof 运算符可以用来决定某对象的类是否实现了接口。

18、heap 和 stack 有什么区别。

栈是一种线形集合, 其添加和删除元素的操作应在同一段完成。栈按照后进先出的方式进行处理。

堆是栈的一个组成元素

19、forward 和 redirect 的区别

forward 是服务器请求资源, 服务器直接访问目标地址的 URL, 把那个 URL 的响应内容读取过来, 然后把这些内容再发给浏览器, 浏览器根本不知道服务器发送的内容是从哪儿来的, 所以它的地址栏中还是原来的地址。

redirect 就是服务端根据逻辑, 发送一个状态码, 告诉浏览器重定向去请求那个地址, 一般来说浏览器会用刚才请求的所有参数重新请求。所以 session, request 参数都可以获取。

20、EJB 与 JAVA BEAN 的区别?

Java Bean 是可复用的组件, 对 Java Bean 并没有严格的规范, 理论上讲, 任何一个 Java 类都可以是一个 Bean。但通常情况下, 由于 Java Bean 是被容器所创建 (如 Tomcat) 的, 所以 Java Bean 应具有一个无参的构造器, 另外, 通常 Java Bean 还要实现 Serializable 接口用于实现 Bean 的持久性。Java Bean 实际上相当于微软 COM 模型中的本地进程内 COM 组件, 它是不能被跨进程访问的。Enterprise Java Bean 相当于 DCOM, 即分布式组件。它是基于 Java 的远程方法调用 (RMI) 技术的, 所以 EJB 可以被远程访问 (跨进程、跨计算机)。但 EJB 必须被布署在诸如 Webspere、WebLogic 这样的容器中, EJB 客户从不直接访问真正的 EJB 组件, 而是通过其容器访问。EJB 容器是 EJB 组件的代理, EJB 组件由容器所创建和管理。客户通过容器来访问真正的 EJB 组件。

21、Static Nested Class 和 Inner Class 的不同。

Static Nested Class 是被声明为静态 (static) 的内部类, 它可以不依赖于外部类实例被实例化。而通常的内部类需要在外部类实例化后才能实例化。

22、JSP 中动态 INCLUDE 与静态 INCLUDE 的区别?

动态 INCLUDE 用 jsp:include 动作实现 <jsp:include page="included.jsp" flush="true" /> 它总是会检查所含文件中的变化, 适合用于包含动态页面, 并且可以带参数。

静态 INCLUDE 用 include 伪码实现, 定不会检查所含文件的变化, 适用于包含静态页面 <%@include file="included.htm" %>

23、什么时候用 assert。

assertion(断言)在软件开发中是一种常用的调试方式,很多开发语言中都支持这种机制。在实现中,assertion 就是在程序中的一条语句,它对一个 boolean 表达式进行检查,一个正确程序必须保证这个 boolean 表达式的值为 true; 如果该值为 false,说明程序已经处于不正确的状态下,系统将给出警告或退出。一般来说,assertion 用于保证程序最基本、关键的正确性。assertion 检查通常在开发和测试时开启。为了提高性能,在软件发布后,assertion 检查通常是关闭的。

24、GC 是什么? 为什么要有 GC?

GC 是垃圾收集的意思 (Garbage Collection),内存处理是编程人员容易出现问题的地方,忘记或者错误的内存回收会导致程序或系统的不稳定甚至崩溃,Java 提供的 GC 功能可以自动监测对象是否超过作用域从而达到自动回收内存的目的,Java 语言没有提供释放已分配内存的显示操作方法。

25、short s1 = 1; s1 = s1 + 1;有什么错? short s1 = 1; s1 += 1;有什么错?

short s1 = 1; s1 = s1 + 1; (s1+1 运算结果是 int 型,需要强制转换类型)

short s1 = 1; s1 += 1; (可以正确编译)

26、Math.round(11.5)等於多少? Math.round(-11.5)等於多少?

Math.round(11.5)==12

Math.round(-11.5)==-11

round 方法返回与参数最接近的长整数,参数加 1/2 后求其 floor.

27、String s = new String("xyz");创建了几个 String Object?

两个

28、设计 4 个线程,其中两个线程每次对 j 增加 1,另外两个线程对 j 每次减少 1。写出程序。

以下程序使用内部类实现线程,对 j 增减的时候没有考虑顺序问题。

```
public class ThreadTest1{
    private int j;
    public static void main(String[] args){
        ThreadTest1 tt=new ThreadTest1();
        Inc inc=tt.new Inc();
        Dec dec=tt.new Dec();
        for(int i=0;i<2;i++){
            Thread t=new Thread(inc);
            t.start();
            t=new Thread(dec);
            t.start();
        }
        private synchronized void inc(){
            j++;
            System.out.println(Thread.currentThread().getName()+"-inc:"+j);
        }
        private synchronized void dec(){
            j--;
            System.out.println(Thread.currentThread().getName()+"-dec:"+j);
        }
    }
}
```

```

    }
    class Inc implements Runnable{
public void run(){
for(int i=0;i<100;i++){
inc();
}
}
    }
    class Dec implements Runnable{
public void run(){
for(int i=0;i<100;i++){
dec();
}
}
    }
}

```

29、Java 有没有 goto?

java 中的保留字，现在没有在 java 中使用。

30、启动一个线程是用 run()还是 start()?

启动一个线程是调用 start()方法，使线程所代表的虚拟处理机处于可运行状态，这意味着它可以由 JVM 调度并执行。这并不意味着线程就会立即运行。run()方法可以产生必须退出的标志来停止一个线程。

31、EJB 包括 (Session Bean Entity Bean) 说出他们的生命周期，及如何管理事务的?

SessionBean: Stateless Session Bean 的生命周期是由容器决定的，当客户机发出请求要建立一个 Bean 的实例时，EJB 容器不一定要创建一个新的 Bean 的实例供客户机调用，而是随便找一个现有的实例提供给客户机。当客户机第一次调用一个 Stateful Session Bean 时，容器必须立即在服务器中创建一个新的 Bean 实例，并关联到客户机上，以后此客户机调用 Stateful Session Bean 的方法时容器会把调用分派到与此客户机相关联的 Bean 实例。

EntityBean: Entity Beans 能存活相对较长的时间，并且状态是持续的。只要数据库中的数据存在，Entity beans 就一直存活。而不是按照应用程序或者服务进程来说的。即使 EJB 容器崩溃了，Entity beans 也是存活的。Entity Beans 生命周期能够被容器或者 Beans 自己管理。

EJB 通过以下技术管理实务:对象管理组织(OMG)的对象实务服务(OTS), Sun Microsystems 的 Transaction Service (JTS)、Java Transaction API (JTA)，开发组 (X/Open) 的 XA 接口。

32、应用服务器有那些?

BEA WebLogic Server, IBM WebSphere Application Server, Oracle9i Application Server, jBoss, Tomcat

33、给我一个你最常见到的 runtime exception。

ArithmeticException, ArrayStoreException, BufferOverflowException,
BufferUnderflowException, CannotRedoException, CannotUndoException, ClassCastException,
CMMException, ConcurrentModificationException, DOMException, EmptyStackException,
IllegalArgumentException, IllegalMonitorStateException, IllegalPathStateException,

IllegalStateException, ImagingOpException, IndexOutOfBoundsException,
 MissingResourceException, NegativeArraySizeException, NoSuchElementException,
 NullPointerException, ProfileDataException, ProviderException, RasterFormatException,
 SecurityException, SystemException, UndeclaredThrowableException,
 UnmodifiableSetException, UnsupportedOperationException

34、接口是否可继承接口？抽象类是否可实现(implements)接口？抽象类是否可继承实体类(concrete class)？

接口可以继承接口。抽象类可以实现(implements)接口，抽象类是否可继承实体类，但前提是实体类必须有明确的构造函数。

35、List, Set, Map 是否继承自 Collection 接口？

List, Set 是，Map 不是

36、说出数据连接池的工作机制是什么？

J2EE 服务器启动时会建立一定数量的池连接，并一直维持不少于此数目的池连接。客户端程序需要连接时，池驱动程序会返回一个未使用的池连接并将其标记为忙。如果当前没有空闲连接，池驱动程序就新建一定数量的连接，新建连接的数量有配置参数决定。当使用的池连接调用完成后，池驱动程序将此连接标记为空闲，其他调用就可以使用这个连接。

37、abstract 的 method 是否可同时是 static,是否可同时是 native, 是否可同时是 synchronized? 都不能

38、数组有没有 length()这个方法? String 有没有 length()这个方法?

数组没有 length()这个方法，有 length 的属性。String 有 length()这个方法。

39、Set 里的元素是不能重复的，那么用什么方法来区分重复与否呢？是用 == 还是 equals()？它们有何区别？

Set 里的元素是不能重复的，那么用 iterator()方法来区分重复与否。equals()是判断两个 Set 是否相等。

equals()和 == 方法决定引用值是否指向同一对象。equals()在类中被覆盖，为的是当两个分离的对象的内容和类型相配的时候，返回 true。

40、构造器 Constructor 是否可被 override?

构造器 Constructor 不能被继承，因此不能重写 Overriding，但可以被重载 Overloading。

41、是否可以继承 String 类？

String 类是 final 类故不可以继承。

42、switch 是否能作用在 byte 上，是否能作用在 long 上，是否能作用在 String 上？

switch (expr1) 中，expr1 是一个整数表达式。因此传递给 switch 和 case 语句的参数应该是 int、short、char 或者 byte。long、string 都不能作用于 switch。

43、try {} 里有一个 return 语句，那么紧跟在这个 try 后的 finally {} 里的 code 会不会被执行，什么时候被执行，在 return 前还是后？

会执行，在 return 前执行。

44、编程题：用最有效率的方法算出 2 乘以 8 等於几？

2 << 3

45、两个对象值相同(x.equals(y) == true)，但却可有不同的 hash code，这句话对不对？

不对，有相同的 hash code。

46、当一个对象被当作参数传递到一个方法后，此方法可改变这个对象的属性，并可返回变化后的结果，那么这里到底是值传递还是引用传递？

是值传递。Java 编程语言只有值传递参数。当一个对象实例作为一个参数被传递到方法中时，参数的值就是对该对象的引用。对象的内容可以在被调用的方法中改变，但对象的引用

是永远不会改变的。

47、当一个线程进入一个对象的一个 `synchronized` 方法后，其它线程是否可进入此对象的其它方法？

不能，一个对象的一个 `synchronized` 方法只能由一个线程访问。

48、编程题：写一个 Singleton 出来。

Singleton 模式主要作用是保证在 Java 应用程序中，一个类 Class 只有一个实例存在。

一般 Singleton 模式通常有几种形式：

第一种形式：定义一个类，它的构造函数为 `private` 的，它有一个 `static` 的 `private` 的该类变量，在类初始化时实例化，通过一个 `public` 的 `getInstance` 方法获取对它的引用，继而调用其中的方法。

```
public class Singleton {
    private Singleton(){}
        //在自己内部定义自己一个实例，是不是很奇怪？
        //注意这是 private 只供内部调用
    private static Singleton instance = new Singleton();
        //这里提供了一个供外部访问本 class 的静态方法，可以直接访问
    public static Singleton getInstance() {
        return instance;
    }
}
```

第二种形式：

```
public class Singleton {
    private static Singleton instance = null;
    public static synchronized Singleton getInstance()
        //这个方法比上面有所改进，不用每次都要进行生成对象，只第一次
        //使用时生成实例，提高了效率！
    if (instance==null)
        instance=new Singleton();
    return instance;
}
```

其他形式：

定义一个类，它的构造函数为 `private` 的，所有方法为 `static` 的。

一般认为第一种形式要更加安全些

49、Java 的接口和 C++ 的虚类的相同和不同处。

由于 Java 不支持多继承，而有可能某个类或对象要使用分别在几个类或对象里面的方法或属性，现有的单继承机制就不能满足要求。与继承相比，接口有更高的灵活性，因为接口中没有任何实现代码。当一个类实现了接口以后，该类要实现接口里面所有的方法和属性，并且接口里面的属性在默认状态下面都是 `public static`，所有方法默认情况下是 `public`。一个类可以实现多个接口。

50、Java 中的异常处理机制的简单原理和应用。

当 JAVA 程序违反了 JAVA 的语义规则时，JAVA 虚拟机就会将发生的错误表示为一个异常。违反语义规则包括 2 种情况。一种是 JAVA 类库内置的语义检查。例如数组下标越界，会引发 `IndexOutOfBoundsException`；访问 `null` 的对象时会引发 `NullPointerException`。另一种情况就是 JAVA 允许程序员扩展这种语义检查，程序员可以创建自己的异常，并自由选择何时用

throw 关键字引发异常。所有的异常都是 java.lang.Throwable 的子类。

51、垃圾回收的优点和原理。并考虑 2 种回收机制。

Java 语言中一个显著的特点就是引入了垃圾回收机制,使 c++程序员最头疼的内存管理的问题迎刃而解,它使得 Java 程序员在编写程序的时候不再需要考虑内存管理。由于有个垃圾回收机制,Java 中的对象不再有"作用域"的概念,只有对象的引用才有"作用域"。垃圾回收可以有效的防止内存泄露,有效的使用可以使用的内存。垃圾回收器通常是作为一个单独的低级别的线程运行,不可预知的情况下对内存堆中已经死亡的或者长时间没有使用的对象进行清楚和回收,程序员不能实时的调用垃圾回收器对某个对象或所有对象进行垃圾回收。回收机制有分代复制垃圾回收和标记垃圾回收,增量垃圾回收。

52、请说出你所知道的线程同步的方法。

wait():使一个线程处于等待状态,并且释放所持有的对象的 lock。

sleep():使一个正在运行的线程处于睡眠状态,是一个静态方法,调用此方法要捕捉 InterruptedException 异常。

notify():唤醒一个处于等待状态的线程,注意的是在调用此方法的时候,并不能确切的唤醒某一个等待状态的线程,而是由 JVM 确定唤醒哪个线程,而且不是按优先级。

Allnotity():唤醒所有处入等待状态的线程,注意并不是给所有唤醒线程一个对象的锁,而是让它们竞争。

53、你所知道的集合类都有哪些?主要方法?

最常用的集合类是 List 和 Map。List 的具体实现包括 ArrayList 和 Vector,它们是可变大小的列表,比较适合构建、存储和操作任何类型对象的元素列表。List 适用于按数值索引访问元素的情形。

Map 提供了一个更通用的元素存储方法。Map 集合类用于存储元素对(称作"键"和"值"),其中每个键映射到一个值。

54、描述一下 JVM 加载 class 文件的原理机制?

JVM 中类的装载是由 ClassLoader 和它的子类来实现的,Java Class Loader 是一个重要的 Java 运行时系统组件。它负责在运行时查找和装入类文件。

55、char 型变量中能不能存贮一个中文汉字?为什么?

能够定义成为一个中文的,因为 java 中以 unicode 编码,一个 char 占 16 个字节,所以放一个中文是没问题的

56、多线程有几种实现方法,都是什么?同步有几种实现方法,都是什么?

多线程有两种实现方法,分别是继承 Thread 类与实现 Runnable 接口

同步的实现方面有两种,分别是 synchronized,wait 与 notify

57、JSP 的内置对象及方法。

request 表示 HttpServletRequest 对象。它包含了有关浏览器请求的信息,并且提供了几个用于获取 cookie,header, 和 session 数据的有用的方法。

response 表示 HttpServletResponse 对象,并提供了几个用于设置送回浏览器的响应的方法(如 cookies,头信息等)

out 对象是 javax.jsp.JspWriter 的一个实例,并提供了几个方法使你能用于向浏览器回送输出结果。

pageContext 表示一个 javax.servlet.jsp.PageContext 对象。它是用于方便存取各种范围的名字空间、servlet 相关的对象的 API,并且包装了通用的 servlet 相关功能的方法。

session 表示一个请求的 javax.servlet.http.HttpSession 对象。Session 可以存贮用户的状态信息

applicaton 表示一个 javax.servle.ServletContext 对象。这有助于查找有关 servlet 引擎和

servlet 环境的信息

config 表示一个 javax.servlet.ServletConfig 对象。该对象用于存取 servlet 实例的初始化参数。

page 表示从该页面产生的一个 servlet 实例

58、线程的基本概念、线程的基本状态以及状态之间的关系

线程指在程序执行过程中,能够执行程序代码的一个执行单位,每个程序至少都有一个线程,也就是程序本身。

Java 中的线程有四种状态分别是: 运行、就绪、挂起、结束。

59、JSP 的常用指令

```
<%@page language="java" contentType="text/html; charset=gb2312" session="true"
buffer="64kb" autoFlush="true" isThreadSafe="true" info="text" errorPage="error.jsp"
isErrorPage="true" isELIgnored="true" pageEncoding="gb2312" import="java.sql.*"%>
```

isErrorPage(是否能使用 Exception 对象), isELIgnored(是否忽略表达式)

```
<%@include file="filename"%>
```

```
<%@taglib prefix="c" uri="http://....."%>
```

60、什么情况下调用 doGet() 和 doPost()?

Jsp 页面中的 form 标签里的 method 属性为 get 时调用 doGet(), 为 post 时调用 doPost()。

61、servlet 的生命周期

web 容器加载 servlet, 生命周期开始。通过调用 servlet 的 init() 方法进行 servlet 的初始化。通过调用 service() 方法实现, 根据请求的不同调用不同的 do***() 方法。结束服务, web 容器调用 servlet 的 destroy() 方法。

62、如何实现 servlet 的单线程模式

```
<%@ page isThreadSafe="false"%>
```

63、页面间对象传递的方法

request, session, application, cookie 等

64、JSP 和 Servlet 有哪些相同点和不同点, 他们之间的联系是什么?

JSP 是 Servlet 技术的扩展, 本质上是 Servlet 的简易方式, 更强调应用的外表表达。JSP 编译后是"类 servlet"。Servlet 和 JSP 最主要的不同点在于, Servlet 的应用逻辑是在 Java 文件中, 并且完全从表示层中的 HTML 里分离开来。而 JSP 的情况是 Java 和 HTML 可以组合成一个扩展名为.jsp 的文件。JSP 侧重于视图, Servlet 主要用于控制逻辑。

65、四种会话跟踪技术

会话作用域 ServletsJSP 页面描述

page 否是代表与一个页面相关的对象和属性。一个页面由一个编译好的 Java servlet 类(可以带有任何的 include 指令, 但是没有 include 动作)表示。这既包括 servlet 又包括被编译成 servlet 的 JSP 页面

request 是代表与 Web 客户机发出的一个请求相关的对象和属性。一个请求可能跨越多个页面, 涉及多个 Web 组件(由于 forward 指令和 include 动作的关系)

session 是代表与用于某个 Web 客户机的一个用户体验相关的对象和属性。一个 Web 会话可以也经常跨越多个客户机请求

application 是代表与整个 Web 应用程序相关的对象和属性。这实质上是跨越整个 Web 应用程序, 包括多个页面、请求和会话的一个全局作用域

66、Request 对象的主要方法:

setAttribute(String name, Object): 设置名字为 name 的 request 的参数值

getAttribute(String name): 返回由 name 指定的属性值

getAttributeNames(): 返回 request 对象所有属性的名字集合, 结果是一个枚举的实例
 get_cookies(): 返回客户端的所有 Cookie 对象, 结果是一个 Cookie 数组
 getCharacterEncoding(): 返回请求中的字符编码方式
 getContentLength(): 返回请求的 Body 的长度
 getHeader(String name): 获得 HTTP 协议定义的文件头信息
 getHeaders(String name): 返回指定名字的 request Header 的所有值, 结果是一个枚举的实例
 getHeaderNames(): 返回所以 request Header 的名字, 结果是一个枚举的实例
 getInputStream(): 返回请求的输入流, 用于获得请求中的数据
 getMethod(): 获得客户端向服务器端传送数据的方法
 getParameter(String name): 获得客户端传送给服务器端的有 name 指定的参数值
 getParameterNames(): 获得客户端传送给服务器端的所有参数的名字, 结果是一个枚举的实例
 getParameterValues(String name): 获得有 name 指定的参数的所有值
 getProtocol(): 获取客户端向服务器端传送数据所依据的协议名称
 getQueryString(): 获得查询字符串
 getRequestURI(): 获取发出请求字符串的客户端地址
 getRemoteAddr(): 获取客户端的 IP 地址
 getRemoteHost(): 获取客户端的名字
 getSession([Boolean create]): 返回和请求相关 Session
 getServerName(): 获取服务器的名字
 getServletPath(): 获取客户端所请求的脚本文件的路径
 getServerPort(): 获取服务器的端口号
 removeAttribute(String name): 删除请求中的一个属性

67、J2EE 是技术还是平台还是框架?

J2EE 本身是一个标准, 一个为企业分布式应用开发提供标准平台。

J2EE 也是一个框架, 包括 JSP、JNF、RMI、JMS、EJB、JTA 等技术。

68、我们在 web 应用开发过程中经常遇到输出某种编码的字符, 如 iso8859-1 等, 如何输出一个某种编码的字符串

```

Public String translate(String str) {
    String tempStr = "";
    try {
        tempStr = new String(str.getBytes("ISO-8859-1"), "GBK");
        tempStr = tempStr.trim();
    }
    catch (Exception e) {
        System.err.println(e.getMessage());
    }
    return tempStr;
}
  
```

69、简述逻辑操作(&,&,&)与条件操作(&&,&&)的区别。

区别主要答两点: a. 条件操作只能操作布尔型的, 而逻辑操作不仅可以操作布尔型, 而且可以操作数值型

b. 逻辑操作不会产生短路

70、XML 文档定义有几种形式? 它们之间有何本质区别? 解析 XML 文档有哪几种方式?

a: 两种形式 dtd schema, b: 本质区别:schema 本身是 xml 的, 可以被 XML 解析器解析(这也是从 DTD 上发展 schema 的根本目的), c:有 DOM,SAX,STAX 等

DOM:处理大型文件时其性能下降的非常厉害。这个问题是由 DOM 的树结构所造成的, 这种结构占用的内存较多, 而且 DOM 必须在解析文件之前把整个文档装入内存,适合对 XML 的随机访问

SAX:不现于 DOM,SAX 是事件驱动型的 XML 解析方式。它顺序读取 XML 文件, 不需要一次全部装载整个文件。当遇到像文件开头, 文档结束, 或者标签开头与标签结束时, 它会触发一个事件, 用户通过在其回调事件中写入处理代码来处理 XML 文件, 适合对 XML 的顺序访问

STAX:Streaming API for XML (StAX)

71、简述 synchronized 和 java.util.concurrent.locks.Lock 的异同 ?

主要相同点: Lock 能完成 synchronized 所实现的所有功能

主要不同点: Lock 有比 synchronized 更精确的线程语义和更好的性能。synchronized 会自动释放锁, 而 Lock 一定要求程序员手工释放, 并且必须在 finally 从句中释放。

72、EJB 的角色和三个对象

一个完整的基于 EJB 的分布式计算结构由六个角色组成, 这六个角色可以由不同的开发商提供, 每个角色所作的工作必须遵循 Sun 公司提供的 EJB 规范, 以保证彼此之间的兼容性。这六个角色分别是 EJB 组件开发者 (Enterprise Bean Provider)、应用组合者 (Application Assembler)、部署者 (Deployer)、EJB 服务器提供者 (EJB Server Provider)、EJB 容器提供者 (EJB Container Provider)、系统管理员 (System Administrator)

三个对象是 Remote (Local) 接口、Home (LocalHome) 接口, Bean 类

73、EJB 容器提供的服务

主要提供声明周期管理、代码产生、持续性管理、安全、事务管理、锁和并发管理等服务。

74、EJB 规范规定 EJB 中禁止的操作有哪些?

1.不能操作线程和线程 API(线程 API 有非线程对象的方法如 notify,wait 等), 2.不能操作 awt, 3.不能实现服务端功能, 4.不能对静态属性存取, 5.不能使用 IO 操作直接存取文件系统, 6.不能加载本地库, 7.不能将 this 作为变量和返回, 8.不能循环调用。

75、remote 接口和 home 接口主要作用

remote 接口定义了业务方法, 用于 EJB 客户端调用业务方法。

home 接口是 EJB 工厂用于创建和移除查找 EJB 实例

76、bean 实例的生命周期

对于 Stateless Session Bean、Entity Bean、Message Driven Bean 一般存在缓冲池管理, 而对于 Entity Bean 和 Statefull Session Bean 存在 Cache 管理, 通常包含创建实例, 设置上下文、创建 EJB Object (create)、业务方法调用、remove 等过程, 对于存在缓冲池管理的 Bean, 在 create 之后实例并不从内存清除, 而是采用缓冲池调度机制不断重用实例, 而对于存在 Cache 管理的 Bean 则通过激活和去激活机制保持 Bean 的状态并限制内存中实例数量。

77、EJB 的激活机制

以 Stateful Session Bean 为例: 其 Cache 大小决定了内存中可以同时存在的 Bean 实例的数量, 根据 MRU 或 NRU 算法, 实例在激活和去激活状态之间迁移, 激活机制是当客户端调用某个 EJB 实例业务方法时, 如果对应 EJB Object 发现自己没有绑定对应的 Bean 实例则从其去激活 Bean 存储中 (通过序列化机制存储实例) 回复 (激活) 此实例。状态变迁前会调用对应的 ejbActive 和 ejbPassivate 方法。

78、EJB 的几种类型

会话 (Session) Bean, 实体 (Entity) Bean 消息驱动的 (Message Driven) Bean

会话 Bean 又可分为有状态 (Stateful) 和无状态 (Stateless) 两种

实体 Bean 可分为 Bean 管理的持续性 (BMP) 和容器管理的持续性 (CMP) 两种

79、客服端调用 EJB 对象的几个基本步骤

设置 JNDI 服务工厂以及 JNDI 服务地址系统属性, 查找 Home 接口, 从 Home 接口调用 Create 方法创建 Remote 接口, 通过 Remote 接口调用其业务方法。

80、如何给 weblogic 指定大小的内存?

在启动 Weblogic 的脚本中 (位于所在 Domain 对应服务器目录下的 startServerName), 增加 set MEM_ARGS=-Xms32m -Xmx200m, 可以调整最小内存为 32M, 最大 200M

81、如何设定的 weblogic 的热启动模式(开发模式)与产品发布模式?

可以在管理控制台中修改对应服务器的启动模式为开发或产品模式之一。或者修改服务的启动文件或者 commenv 文件, 增加 set PRODUCTION_MODE=true。

82、如何启动时不需输入用户名与密码?

修改服务启动文件, 增加 WLS_USER 和 WLS_PW 项。也可以在 boot.properties 文件中增加加密过的用户名和密码。

83、在 weblogic 管理制台中对一个应用域(或者说是一个网站,Domain)进行 jms 及 ejb 或连接池等相关信息进行配置后,实际保存在什么文件中?

保存在此 Domain 的 config.xml 文件中, 它是服务器的核心配置文件。

84、说说 weblogic 中一个 Domain 的缺省目录结构?比如要将一个简单的 helloWorld.jsp 放入何目录下,然的在浏览器上就可打入 http://主机:端口号/helloword.jsp 就可以看到运行结果了? 又比如这其中用到了一个自己写的 javaBean 该如何办?

Domain 目录服务器目录 applications, 将应用目录放在此目录下将可以作为应用访问, 如果是 Web 应用, 应用目录需要满足 Web 应用目录要求, jsp 文件可以直接放在应用目录中, Javabeen 需要放在应用目录的 WEB-INF 目录的 classes 目录下。设置服务器的缺省应用将可以实现浏览器上无需输入应用名。

85、在 weblogic 中发布 ejb 需涉及到哪些配置文件?

不同类型的 EJB 涉及的配置文件不同, 都涉及到的配置文件包括 ejb-jar.xml, weblogic-ejb-jar.xml。CMP 实体 Bean 一般还需要 weblogic-cmp-rdbms-jar.xml

86、如何在 weblogic 中进行 ssl 配置与客户端的认证配置或说说 j2ee(标准)进行 ssl 的配置缺省安装中使用 DemoIdentity.jks 和 DemoTrust.jks KeyStore 实现 SSL, 需要配置服务器使用 Enable SSL, 配置其端口, 在产品模式下需要从 CA 获取私有密钥和数字证书, 创建 identity 和 trust keystore, 装载获得的密钥和数字证书。可以配置此 SSL 连接是单向还是双向的。

87、如何查看在 weblogic 中已经发布的 EJB?

可以使用管理控制台, 在它的 Deployment 中可以查看所有已发布的 EJB

88、CORBA 是什么?用途是什么?

CORBA 标准是公共对象请求代理结构(Common Object Request Broker Architecture), 由对象管理组织 (Object Management Group, 缩写为 OMG)标准化。它的组成是接口定义语言(IDL), 语言绑定(binding;也译为联编)和允许应用程序间互操作的协议。其目的为: 用不同的程序设计语言书写在不同的进程中运行, 为不同的操作系统开发。

89、说说你所熟悉或听说过的 j2ee 中的几种常用模式?及对设计模式的一些看法

Session Facade Pattern: 使用 SessionBean 访问 EntityBean

Message Facade Pattern: 实现异步调用

EJB Command Pattern: 使用 Command JavaBeans 取代 SessionBean, 实现轻量级访问

Data Transfer Object Factory: 通过 DTO Factory 简化 EntityBean 数据提供特性

Generic Attribute Access: 通过 AttributeAccess 接口简化 EntityBean 数据提供特性

Business Interface: 通过远程（本地）接口和 Bean 类实现相同接口规范业务逻辑一致性

EJB 架构的设计好坏将直接影响系统的性能、可扩展性、可维护性、组件可重用性及开发效率。项目越复杂，项目队伍越庞大则越能体现良好设计的重要性。

90、说说在 weblogic 中开发消息 Bean 时的 persistent 与 non-persistent 的差别

persistent 方式的 MDB 可以保证消息传递的可靠性,也就是如果 EJB 容器出现问题而 JMS 服务器依然会将消息在此 MDB 可用的时候发送过来,而 non-persistent 方式的消息将被丢弃。

91、Servlet 执行时一般实现哪几个方法？

```
public void init(ServletConfig config)
public ServletConfig getServletConfig()
public String getServletInfo()
public void service(ServletRequest request,ServletResponse response)
public void destroy()
```

92、j2ee 常用的设计模式？说明工厂模式。

Java 中的 23 种设计模式：

Factory（工厂模式）， Builder（建造模式）， Factory Method（工厂方法模式），
 Prototype（原始模型模式）， Singleton（单例模式）， Facade（门面模式），
 Adapter（适配器模式）， Bridge（桥梁模式）， Composite（合成模式），
 Decorator（装饰模式）， Flyweight（享元模式）， Proxy（代理模式），
 Command（命令模式）， Interpreter（解释器模式）， Visitor（访问者模式），
 Iterator（迭代子模式）， Mediator（调停者模式）， Memento（备忘录模式），
 Observer（观察者模式）， State（状态模式）， Strategy（策略模式），
 Template Method（模板方法模式）， Chain Of Responsibility（责任链模式）

工厂模式：工厂模式是一种经常被使用到的模式。根据工厂模式实现的类可以根据提供的数
 据生成一组类中某一个类的实例，通常这一组类有一个公共的抽象父类并且实现了相同的方法，但是这些方法针对不同的数据进行了不同的操作。首先需要一个基类，该类的子类
 通过不同的方法实现基类中的方法。然后需要定义一个工厂类，工厂类可以根据条件生成
 不同的子类实例。当得到子类的实例后，开发人员可以调用基类中的方法而不必考虑到底返
 回的是哪一个子类的实例。

93、EJB 需直接实现它的业务接口或 Home 接口吗，请简述理由。

远程接口和 Home 接口不需要直接实现，他们的实现代码是由服务器产生的，程序运行中对
 应实现类会作为对应接口类型的实例被使用。

94、排序都有哪几种方法？请列举。用 JAVA 实现一个快速排序。

排序的方法有：插入排序（直接插入排序、希尔排序），交换排序（冒泡排序、快速排
 序），选择排序（直接选择排序、堆排序），归并排序，分配排序（箱排序、基数排序）

快速排序的伪代码。

//使用快速排序方法对 a[0:n-1]排序

从 a[0:n-1]中选择一个元素作为 middle，该元素为支点

把余下的元素分割为两段 left 和 right，使得 left 中的元素都小于等于支点，而 right 中
 的元素都大于等于支点

递归地使用快速排序方法对 left 进行排序

递归地使用快速排序方法对 right 进行排序

所得结果为 left+middle+right

95、请对以下在 J2EE 中常用的名词进行解释(或简单描述)

web 容器：给处于其中的应用程序组件（JSP，SERVLET）提供一个环境，使 JSP,SERVLET

直接更改容器中的环境变量接口交互，不必关注其它系统问题。主要有 WEB 服务器来实现。例如：TOMCAT,WEBLOGIC,WEBSPPHERE 等。该容器提供的接口严格遵守 J2EE 规范中的 WEB APPLICATION 标准。我们把遵守以上标准的 WEB 服务器就叫做 J2EE 中的 WEB 容器。

EJB 容器：Enterprise java bean 容器。更具有行业领域特色。他提供给运行在其中的组件 EJB 各种管理功能。只要满足 J2EE 规范的 EJB 放入该容器，马上就会被容器进行高效率的管理。并且可以通过现成的接口来获得系统级别的服务。例如邮件服务、事务管理。

JNDI：(Java Naming & Directory Interface) JAVA 命名目录服务。主要提供的功能是：提供一个目录系统，让其它各地的应用程序在其上面留下自己的索引，从而满足快速查找和定位分布式应用程序的功能。

JMS：(Java Message Service) JAVA 消息服务。主要实现各个应用程序之间的通讯。包括点对点 and 广播。

JTA：(Java Transaction API) JAVA 事务服务。提供各种分布式事务服务。应用程序只需调用其提供的接口即可。

JAF：(Java Action FrameWork) JAVA 安全认证框架。提供一些安全控制方面的框架。让开发者通过各种部署和自定义实现自己的个性安全控制策略。

RMI/IIOP：(Remote Method Invocation /internet 对象请求中介协议) 他们主要用于通过远程调用服务。例如，远程有一台计算机上运行一个程序，它提供股票分析服务，我们可以在本地计算机上实现对其直接调用。当然这是要通过一定的规范才能在异构的系统之间进行通信。RMI 是 JAVA 特有的。

96、JAVA 语言如何进行异常处理，关键字：throws,throw,try,catch,finally 分别代表什么意义？在 try 块中可以抛出异常吗？

Java 通过面向对象的方法进行异常处理，把各种不同的异常进行分类，并提供了良好的接口。在 Java 中，每个异常都是一个对象，它是 Throwable 类或其它子类的实例。当一个方法出现异常后便抛出一个异常对象，该对象中包含有异常信息。调用这个方法可以捕获到这个异常并进行处理。Java 的异常处理是通过 5 个关键词来实现的：try、catch、throw、throws 和 finally。一般情况下是用 try 来执行一段程序，如果出现异常，系统会抛出 (throws) 一个异常，这时候你可以通过它的类型来捕捉 (catch) 它，或最后 (finally) 由缺省处理器来处理。

用 try 来指定一块预防所有"异常"的程序。紧跟在 try 程序后面，应包含一个 catch 子句来指定你想要捕捉的"异常"的类型。

throw 语句用来明确地抛出一个"异常"。

throws 用来标明一个成员函数可能抛出的各种"异常"。

Finally 为确保一段代码不管发生什么"异常"都被执行一段代码。

可以在一个成员函数调用的外面写一个 try 语句，在这个成员函数内部写另一个 try 语句保护其他代码。每当遇到一个 try 语句，"异常"的框架就放到堆栈上面，直到所有的 try 语句都完成。如果下一级的 try 语句没有对某种"异常"进行处理，堆栈就会展开，直到遇到有处理这种"异常"的 try 语句。

97、一个".java"源文件中是否可以包括多个类（不是内部类）？有什么限制？

可以。必须只有一个类名与文件名相同。

98、MVC 的各个部分都有那些技术来实现?如何实现？

MVC 是 Model—View—Controller 的简写。"Model" 代表的是应用的业务逻辑（通过 JavaBean, EJB 组件实现），"View" 是应用的表示面（由 JSP 页面产生），"Controller" 是提供应用的处理过程控制（一般是一个 Servlet），通过这种设计模型把应用逻辑，处理过程

和显示逻辑分成不同的组件实现。这些组件可以进行交互和重用。

99、java 中有几种方法可以实现一个线程？用什么关键字修饰同步方法？stop()和 suspend()方法为何不推荐使用？

有两种实现方法，分别是继承 Thread 类与实现 Runnable 接口

用 synchronized 关键字修饰同步方法

反对使用 stop()，是因为它不安全。它会解除由线程获取的所有锁定，而且如果对象处于一种不连贯状态，那么其他线程能在那种状态下检查和修改它们。结果很难检查出真正的问题所在。suspend()方法容易发生死锁。调用 suspend()的时候，目标线程会停下来，但却仍然持有在这之前获得的锁定。此时，其他任何线程都不能访问锁定的资源，除非被"挂起"的线程恢复运行。对任何线程来说，如果它们想恢复目标线程，同时又试图使用任何一个锁定的资源，就会造成死锁。所以不应该使用 suspend()，而应在自己的 Thread 类中置入一个标志，指出线程应该活动还是挂起。若标志指出线程应该挂起，便用 wait()命其进入等待状态。若标志指出线程应当恢复，则用一个 notify()重新启动线程。

100、java 中有几种类型的流？JDK 为每种类型的流提供了一些抽象类以供继承，请说出他们分别是哪些类？

字节流，字符流。字节流继承于 InputStream OutputStream，字符流继承于 InputStreamReader OutputStreamWriter。在 java.io 包中还有许多其他的流，主要是为了提高性能和使用方便。

101、java 中会存在内存泄漏吗，请简单描述。

会。如：int i,i2; return (i-i2); //when i 为足够大的正数,i2 为足够大的负数。结果会造成溢出，导致错误。

102、java 中实现多态的机制是什么？

方法的重写 Overriding 和重载 Overloading 是 Java 多态性的不同表现。重写 Overriding 是父类与子类之间多态性的一种表现，重载 Overloading 是一个类中多态性的一种表现。

103、垃圾回收器的基本原理是什么？垃圾回收器可以马上回收内存吗？有什么办法主动通知虚拟机进行垃圾回收？

对于 GC 来说，当程序创建对象时，GC 就开始监控这个对象的地址、大小以及使用情况。通常，GC 采用有向图的方式记录和管理堆(heap)中的所有对象。通过这种方式确定哪些对象是"可达的"，哪些对象是"不可达的"。当 GC 确定一些对象为"不可达"时，GC 就有责任回收这些内存空间。程序员可以手动执行 System.gc()，通知 GC 运行，但是 Java 语言规范并不保证 GC 一定会执行。

104、静态变量和实例变量的区别？

static i = 10; //常量

class A a; a.i=10; //可变

105、什么是 java 序列化，如何实现 java 序列化？

序列化就是一种用来处理对象流的机制，所谓对象流也就是将对象的内容进行流化。可以对流化后的对象进行读写操作，也可将流化后的对象传输于网络之间。序列化是为了解决在对对象流进行读写操作时所引发的问题。

序列化的实现：将需要被序列化的类实现 Serializable 接口，该接口没有需要实现的方法，implements Serializable 只是为了标注该对象是可被序列化的，然后使用一个输出流(如：FileOutputStream)来构造一个 ObjectOutputStream(对象流)对象，接着，使用 ObjectOutputStream 对象的 writeObject(Object obj)方法就可以将参数为 obj 的对象写出(即保存其状态)，要恢复的话则用输入流。

106、是否可以从一个 static 方法内部发出对非 static 方法的调用？

不可以,如果其中包含对象的 method(); 不能保证对象初始化.

107、写 clone()方法时，通常都有一行代码，是什么？

Clone 有缺省行为，super.clone();他负责产生正确大小的空间，并逐位复制。

108、在 JAVA 中，如何跳出当前的多重嵌套循环？

用 break; return 方法。

109、List、Map、Set 三个接口，存取元素时，各有什么特点？

List 以特定次序来持有元素，可有重复元素。Set 无法拥有重复元素,内部排序。Map 保存 key-value 值，value 可多值。

110、J2EE 是什么？

J2EE 是 Sun 公司提出的多层(multi-tiered),分布式(distributed),基于组件(component-base)的企业级应用模型 (enterprise application model),在这样的一个应用系统中，可按照功能划分为不同的组件，这些组件又可在不同计算机上，并且处于相应的层次(tier)中。所属层次包括客户层(client tier)组件,web 层和组件,Business 层和组件,企业信息系统(EIS)层。

111、UML 方面

标准建模语言 UML。用例图,静态图(包括类图、对象图和包图),行为图,交互图(顺序图,合作图),实现图。

112、说出一些常用的类，包，接口，请各举 5 个

常用的类：BufferedReader BufferedWriter FileReader FileWriter String Integer

常用的包：java.lang java.awt java.io java.util java.sql

常用的接口：Remote List Map Document NodeList

113、开发中都用到了那些设计模式?用在什么场合？

每个模式都描述了一个在我们的环境中不断出现的问题，然后描述了该问题的解决方案的核心。通过这种方式，你可以无数次地使用那些已有的解决方案，无需重复重复相同的工作。主要用到了 MVC 的设计模式。用来开发 JSP/Servlet 或者 J2EE 的相关应用。简单工厂模式等。

114、jsp 有哪些动作?作用分别是什么？

JSP 共有以下 6 种基本动作 jsp:include: 在页面被请求的时候插入一个文件。jsp:useBean: 寻找或者实例化一个 JavaBean。jsp:setProperty: 设置 JavaBean 的属性。jsp:getProperty: 输出某个 JavaBean 的属性。jsp:forward: 把请求转到一个新的页面。jsp:plugin: 根据浏览器类型为 Java 插件生成 OBJECT 或 EMBED 标记。

115、Anonymous Inner Class (匿名内部类) 是否可以 extends(继承)其它类，是否可以 implements(实现)interface(接口)？

可以继承其他类或完成其他接口，在 swing 编程中常用此方式。

116、应用服务器与 WEB SERVER 的区别？

应用服务器：Weblogic、Tomcat、Jboss

WEB SERVER: IIS、Apache

117、BS 与 CS 的联系与区别。

C/S 是 Client/Server 的缩写。服务器通常采用高性能的 PC、工作站或小型机，并采用大型数据库系统，如 Oracle、Sybase、Informix 或 SQL Server。客户端需要安装专用的客户端软件。

B/S 是 Browser/Server 的缩写，客户机上只要安装一个浏览器(Browser)，如 Netscape Navigator 或 Internet Explorer，服务器安装 Oracle、Sybase、Informix 或 SQL Server 等数据库。在这种结构下，用户界面完全通过 WWW 浏览器实现，一部分事务逻辑在前端实现，但是主要事务逻辑在服务器端实现。浏览器通过 Web Server 同数据库进行数据交互。

C/S 与 B/S 区别：

1. 硬件环境不同：

C/S 一般建立在专用的网络上,小范围里的网络环境,局域网之间再通过专门服务器提供连接和数据交换服务。

B/S 建立在广域网之上的,不必是专门的网络硬件环境,例与电话上网,租用设备.信息自己管理.有比 C/S 更强的适应范围,一般只要有操作系统和浏览器就行

2. 对安全要求不同

C/S 一般面向相对固定的用户群,对信息安全的控制能力很强.一般高度机密的信息系统采用 C/S 结构适宜.可以通过 B/S 发布部分可公开信息.

B/S 建立在广域网之上,对安全的控制能力相对弱,可能面向不可知的用户。

3. 对程序架构不同

C/S 程序可以更加注重流程,可以对权限多层次校验,对系统运行速度可以较少考虑.

B/S 对安全以及访问速度的多重的考虑,建立在需要更加优化的基础之上.比 C/S 有更高的要求 B/S 结构的程序架构是发展的趋势,从 MS 的 .Net 系列的 BizTalk 2000 Exchange 2000 等,全面支持网络的构件搭建的系统. SUN 和 IBM 推的 JavaBean 构件技术等,使 B/S 更加成熟.

4. 软件重用不同

C/S 程序可以不可避免的整体性考虑,构件的重用性不如在 B/S 要求下的构件的重用性好.

B/S 对的多重结构,要求构件相对独立的功能.能够相对较好的重用.就象买来的餐桌可以再利用,而不是做在墙上的石头桌子

5. 系统维护不同

C/S 程序由于整体性,必须整体考察,处理出现的问题以及系统升级.升级难.可能是再做一个全新的系统

B/S 构件组成,方面构件个别的更换,实现系统的无缝升级.系统维护开销减到最小.用户从网上自己下载安装就可以实现升级.

6. 处理问题不同

C/S 程序可以处理用户面很窄,并且在相同区域,安全要求高需求,与操作系统相关.应该都是相同的系统

B/S 建立在广域网上,面向不同的用户群,分散地域,这是 C/S 无法作到的.与操作系统平台关系最小.

7. 用户接口不同

C/S 多是建立的 Window 平台上,表现方法有限,对程序员普遍要求较高

B/S 建立在浏览器上,有更加丰富和生动的表现方式与用户交流.并且大部分难度减低,减低开发成本.

8. 信息流不同

C/S 程序一般是典型的中央集权的机械式处理,交互性相对低

B/S 信息流向可变化, B-B B-C B-G 等信息、流向的变化,更像交易中心。

118、LINUX 下线程, GDI 类的解释。

LINUX 实现的就是基于核心轻量级进程的"一对一"线程模型,一个线程实体对应一个核心轻量级进程,而线程之间的管理在核外函数库中实现。

GDI 类为图像设备编程接口类库。

119、STRUTS 的应用(如 STRUTS 架构)

Struts 是采用 Java Servlet/JavaServer Pages 技术,开发 Web 应用程序的开放源码的 framework。采用 Struts 能开发出基于 MVC(Model-View-Controller)设计模式的应用构架。Struts 有如下的主要功能:一.包含一个 controller servlet,能将用户的请求发送到相应的 Action

对象。二.JSP 自由 tag 库, 并且在 controller servlet 中提供关联支持, 帮助开发人员创建交互式表单应用。三.提供了一系列实用对象: XML 处理、通过 Java reflection APIs 自动处理 JavaBeans 属性、国际化的提示和消息。

120、Jdo 是什么?

JDO 是 Java 对象持久化的新的规范, 为 java data object 的简称, 也是一个用于存取某种数据仓库中的对象的标准化 API。JDO 提供了透明的对象存储, 因此对开发人员来说, 存储数据对象完全不需要额外的代码 (如 JDBC API 的使用)。这些繁琐的例行工作已经转移到 JDO 产品提供商身上, 使开发人员解脱出来, 从而集中时间和精力在业务逻辑上。另外, JDO 很灵活, 因为它可以在任何数据底层上运行。JDBC 只是面向关系数据库 (RDBMS) JDO 更通用, 提供到任何数据底层的存储功能, 比如关系数据库、文件、XML 以及对象数据库 (ODBMS) 等等, 使得应用可移植性更强。

121、内部类可以引用他包含类的成员吗? 有没有什么限制?

一个内部类对象可以访问创建它的外部类对象的内容

122、WEB SERVICE 名词解释。JSDDL 开发包的介绍。JAXP、JAXM 的解释。SOAP、UDDI,WSDL 解释。

Web Service Web Service 是基于网络的、分布式的模块化组件, 它执行特定的任务, 遵守具体的技术规范, 这些规范使得 Web Service 能与其他兼容的组件进行互操作。

JAXP(Java API for XML Parsing) 定义了使用 DOM, SAX, XSLT 的通用的接口。这样在你的程序中你只要使用这些通用的接口, 当你需要改变具体的实现时候也不需要修改代码。

JAXM(Java API for XML Messaging) 是为 SOAP 通信提供访问方法和传输机制的 API。

WSDL 是一种 XML 格式, 用于将网络服务描述为一组端点, 这些端点对包含面向文档信息或面向过程信息的信息进行操作。这种格式首先对操作和信息进行抽象描述, 然后将其绑定到具体的网络协议和消息格式上以定义端点。相关的具体端点组合成为抽象端点 (服务)。

SOAP 即简单对象访问协议 (Simple Object Access Protocol), 它是用于交换 XML 编码信息的轻量级协议。

UDDI 的目的是为电子商务建立标准, UDDI 是一套基于 Web 的、分布式的、为 Web Service 提供的、信息注册中心的实现标准规范, 同时也包含一组使企业能将自身提供的 Web Service 注册, 以使别的企业能够发现的访问协议的实现标准。

JAVA 代码查错

1.

```
abstract class Name {
    private String name;
    public abstract boolean isStupidName(String name) {}
}
```

大侠们, 这有何错误?

答案: 错。abstract method 必须以分号结尾, 且不带花括号。

2.

```
public class Something {
    void doSomething () {
        private String s = "";
```

```

        int l = s.length();
    }
}

```

有错吗?

答案: 错。局部变量前不能放置任何访问修饰符 (private, public, 和 protected)。final 可以用来修饰局部变量

(final 如同 abstract 和 strictfp, 都是非访问修饰符, strictfp 只能修饰 class 和 method 而非 variable)。

3.

```

abstract class Something {
    private abstract String doSomething ();
}

```

这好像没什么错吧?

答案: 错。abstract 的 methods 不能以 private 修饰。abstract 的 methods 就是让子类 implement(实现)具体细节的, 怎么可以用 private 把 abstract method 封锁起来呢? (同理, abstract method 前不能加 final)。

4.

```

public class Something {
    public int addOne(final int x) {
        return ++x;
    }
}

```

这个比较明显。

答案: 错。int x 被修饰成 final, 意味着 x 不能在 addOne method 中被修改。

5.

```

public class Something {
    public static void main(String[] args) {
        Other o = new Other();
        new Something().addOne(o);
    }
    public void addOne(final Other o) {
        o.i++;
    }
}

```

```

class Other {
    public int i;
}

```

和上面的很相似, 都是关于 final 的问题, 这有错吗?

答案: 正确。在 addOne method 中, 参数 o 被修饰成 final。如果在 addOne method 里我们修改了 o 的 reference

(比如: o = new Other();), 那么如同上例这题也是错的。但这里修改的是 o 的 member variable (成员变量), 而 o 的 reference 并没有改变。

6.

```

class Something {

```

```

int i;
public void doSomething() {
    System.out.println("i = " + i);
}
}

```

有什么错呢？看不出来啊。

答案：正确。输出的是*"i = 0"*。int i 属于 instant variable (实例变量，或叫成员变量)。instant variable 有 default value。int 的 default value 是 0。

7.

```

class Something {
    final int i;
    public void doSomething() {
        System.out.println("i = " + i);
    }
}

```

和上面一题只有一个地方不同，就是多了一个 final。这难道就错了吗？

答案：错。final int i 是个 final 的 instant variable (实例变量，或叫成员变量)。final 的 instant variable 没有 default value，必须在 constructor (构造器)结束之前被赋予一个明确的值。可以修改为*"final int i = 0;"*。

8.

```

public class Something {
    public static void main(String[] args) {
        Something s = new Something();
        System.out.println("s.doSomething() returns " + doSomething());
    }
    public String doSomething() {
        return "Do something. ";
    }
}

```

看上去很完美。

答案：错。看上去在 main 里 call doSomething 没有什么问题，毕竟两个 methods 都在同一个 class 里。但仔细看，main 是 static 的。static method 不能直接 call non-static methods。可改成*"System.out.println("s.doSomething() returns " + s.doSomething());"*。同理，static method 不能访问 non-static instant variable。

9.

此处，Something 类的文件名叫 OtherThing.java

```

class Something {
    private static void main(String[] something_to_do) {
        System.out.println("Do something ...");
    }
}

```

这个好像很明显。

答案：正确。从来没有人说过 Java 的 Class 名字必须和其文件名相同。但 public class 的名字必须和文件名相同。

10.

```

interface A{
    int x = 0;
}
class B{
    int x =1;
}
class C extends B implements A {
    public void pX(){
        System.out.println(x);
    }
    public static void main(String[] args) {
        new C().pX();
    }
}

```

答案：错误。在编译时会发生错误(错误描述不同的 JVM 有不同的信息，意思就是未明确的 x 调用，两个 x 都匹配(就象在同时 import java.util 和 java.sql 两个包时直接声明 Date 一样)。对于父类的变量,可以用 super.x 来明确，而接口的属性默认隐含为 public static final.所以可以通过 A.x 来明确。

11.

```

interface Playable {
    void play();
}
interface Bounceable {
    void play();
}
interface Rollable extends Playable, Bounceable {
    Ball ball = new Ball("PingPong");
}
class Ball implements Rollable {
    private String name;
    public String getName() {
        return name;
    }
    public Ball(String name) {
        this.name = name;
    }
    public void play() {
        ball = new Ball("Football");
        System.out.println(ball.getName());
    }
}

```

这个错误不容易发现。

答案：错。"interface Rollable extends Playable, Bounceable"没有问题。interface 可继承多个

interfaces, 所以这里没错。问题出在 interface Rollable 里的 "Ball ball = new Ball("PingPang");"。任何在 interface 里声明的 interface variable (接口变量, 也可称成员变量), 默认为 public static final。也就是说 "Ball ball = new Ball("PingPang");" 实际上是 "public static final Ball ball = new Ball("PingPang");"。在 Ball 类的 Play() 方法中, "ball = new Ball("Football");" 改变了 ball 的 reference, 而这里的 ball 来自 Rollable interface, Rollable interface 里的 ball 是 public static final 的, final 的 object 是不能被改变 reference 的。因此编译器将在 "ball = new Ball("Football");" 这里显示有错。

JAVA 编程题

1. 现在输入 n 个数字, 以逗号, 分开; 然后可选择升或者降序排序; 按提交键就在另一页面显示按什么排序, 结果为, 提供 reset

```
import java.util.*;

public class bycomma{
    public static String[] splitStringByComma(String source){
        if(source==null||source.trim().equals(""))
            return null;
        StringTokenizer commaToker = new StringTokenizer(source,",");
        String[] result = new String[commaToker.countTokens()];
        int i=0;
        while(commaToker.hasMoreTokens()){
            result[i] = commaToker.nextToken();
            i++;
        }
        return result;
    }
    public static void main(String args[]){
        String[] s = splitStringByComma("1,2,3,4,3,9,1");
        int[] ii = new int[s.length];
        for(int i = 0;i<s.length;i++){
            ii[i] =Integer.parseInt(s[i]);
        }
        Arrays.sort(ii);
        //asc
        for(int i=0;i<s.length;i++){
            System.out.println(ii[i]);
        }
        //desc
        for(int i=(s.length-1);i>=0;i--){
            System.out.println(ii[i]);
        }
    }
}
```

2. 金额转换, 阿拉伯数字的金额转换成中国传统的形式如: (¥1011) → (一千零一拾一

元整) 输出。

```
package test.format;
import java.text.NumberFormat;
import java.util.HashMap;
public class SimpleMoneyFormat {
    public static final String EMPTY = "";
    public static final String ZERO = "零";
    public static final String ONE = "壹";
    public static final String TWO = "贰";
    public static final String THREE = "叁";
    public static final String FOUR = "肆";
    public static final String FIVE = "伍";
    public static final String SIX = "陆";
    public static final String SEVEN = "柒";
    public static final String EIGHT = "捌";
    public static final String NINE = "玖";
    public static final String TEN = "拾";
    public static final String HUNDRED = "佰";
    public static final String THOUSAND = "仟";
    public static final String TEN_THOUSAND = "万";
    public static final String HUNDRED_MILLION = "亿";
    public static final String YUAN = "元";
    public static final String JIAO = "角";
    public static final String FEN = "分";
    public static final String DOT = ".";
    private static SimpleMoneyFormat formatter = null;
    private HashMap chineseNumberMap = new HashMap();
    private HashMap chineseMoneyPattern = new HashMap();
    private NumberFormat numberFormat = NumberFormat.getInstance();
    private SimpleMoneyFormat() {
        numberFormat.setMaximumFractionDigits(4);
        numberFormat.setMinimumFractionDigits(2);
        numberFormat.setGroupingUsed(false);
        chineseNumberMap.put("0", ZERO);
        chineseNumberMap.put("1", ONE);
        chineseNumberMap.put("2", TWO);
        chineseNumberMap.put("3", THREE);
        chineseNumberMap.put("4", FOUR);
        chineseNumberMap.put("5", FIVE);
        chineseNumberMap.put("6", SIX);
        chineseNumberMap.put("7", SEVEN);
        chineseNumberMap.put("8", EIGHT);
        chineseNumberMap.put("9", NINE);
        chineseNumberMap.put(DOT, DOT);
    }
}
```

```

    chineseMoneyPattern.put("1", TEN);
    chineseMoneyPattern.put("2", HUNDRED);
    chineseMoneyPattern.put("3", THOUSAND);
    chineseMoneyPattern.put("4", TEN_THOUSAND);
    chineseMoneyPattern.put("5", TEN);
    chineseMoneyPattern.put("6", HUNDRED);
    chineseMoneyPattern.put("7", THOUSAND);
    chineseMoneyPattern.put("8", HUNDRED_MILLION);
}

public static SimpleMoneyFormat getInstance() {
    if (formatter == null)
        formatter = new SimpleMoneyFormat();
    return formatter;
}

public String format(String moneyStr) {
    checkPrecision(moneyStr);
    String result;
    result = convertToChineseNumber(moneyStr);
    result = addUnitsToChineseMoneyString(result);
    return result;
}

public String format(double moneyDouble) {
    return format(numberFormat.format(moneyDouble));
}

public String format(int moneyInt) {
    return format(numberFormat.format(moneyInt));
}

public String format(long moneyLong) {
    return format(numberFormat.format(moneyLong));
}

public String format(Number moneyNum) {
    return format(numberFormat.format(moneyNum));
}

private String convertToChineseNumber(String moneyStr) {
    String result;
    StringBuffer cMoneyStringBuffer = new StringBuffer();
    for (int i = 0; i < moneyStr.length(); i++) {
        cMoneyStringBuffer.append(chineseNumberMap.get(moneyStr.substring(i, i + 1)));
    }
    //拾佰仟万亿等都是汉字里面才有的单位，加上它们
    int indexOfDot = cMoneyStringBuffer.indexOf(DOT);
    int moneyPatternCursor = 1;
    for (int i = indexOfDot - 1; i > 0; i--) {
        cMoneyStringBuffer.insert(i, chineseMoneyPattern.get(EMPTY + moneyPatternCursor));
    }
}

```

```

        moneyPatternCursor = moneyPatternCursor == 8 ? 1 : moneyPatternCursor + 1;
    }

    String fractionPart = cMoneyStringBuffer.substring(cMoneyStringBuffer.indexOf("."));
    cMoneyStringBuffer.delete(cMoneyStringBuffer.indexOf("."), cMoneyStringBuffer.length());
    while (cMoneyStringBuffer.indexOf("零拾") != -1) {
        cMoneyStringBuffer.replace(cMoneyStringBuffer.indexOf("零拾"),
cMoneyStringBuffer.indexOf("零拾") + 2, ZERO);
    }
    while (cMoneyStringBuffer.indexOf("零佰") != -1) {
        cMoneyStringBuffer.replace(cMoneyStringBuffer.indexOf("零佰"),
cMoneyStringBuffer.indexOf("零佰") + 2, ZERO);
    }
    while (cMoneyStringBuffer.indexOf("零仟") != -1) {
        cMoneyStringBuffer.replace(cMoneyStringBuffer.indexOf("零仟"),
cMoneyStringBuffer.indexOf("零仟") + 2, ZERO);
    }
    while (cMoneyStringBuffer.indexOf("零万") != -1) {
        cMoneyStringBuffer.replace(cMoneyStringBuffer.indexOf("零万"),
cMoneyStringBuffer.indexOf("零万") + 2, TEN_THOUSAND);
    }
    while (cMoneyStringBuffer.indexOf("零亿") != -1) {
        cMoneyStringBuffer.replace(cMoneyStringBuffer.indexOf("零亿"),
cMoneyStringBuffer.indexOf("零亿") + 2, HUNDRED_MILLION);
    }
    while (cMoneyStringBuffer.indexOf("零零") != -1) {
        cMoneyStringBuffer.replace(cMoneyStringBuffer.indexOf("零零"),
cMoneyStringBuffer.indexOf("零零") + 2, ZERO);
    }
    if (cMoneyStringBuffer.lastIndexOf(ZERO) == cMoneyStringBuffer.length() - 1)
        cMoneyStringBuffer.delete(cMoneyStringBuffer.length() - 1,
cMoneyStringBuffer.length());
    cMoneyStringBuffer.append(fractionPart);
    result = cMoneyStringBuffer.toString();
    return result;
}

private String addUnitsToChineseMoneyString(String moneyStr) {
    String result;
    StringBuffer cMoneyStringBuffer = new StringBuffer(moneyStr);
    int indexOfDot = cMoneyStringBuffer.indexOf(DOT);
    cMoneyStringBuffer.replace(indexOfDot, indexOfDot + 1, YUAN);

```

面试时候经常会问的一些问题(不断补充中)

面试必备基础题目(虽然不一定常用, 仅用于面试, 面试就是把人搞的都不会然后砍价, 当然您可以讲我可以查资料完成, 但是面试的时候就是没道理的, 起码我是经常看到这些题).

如何把一段逗号分割的字符串转换成一个数组?

request.getAttribute() 和 request.getParameter() 有何区别?

response.sendRedirect() 和 forward() 区别?

<%@include file="xxx.jsp"%>

和 <jsp:include> 区别?

List 和 Map 区别?

Struts 和 Spring 自动填充表单参数到 Bean 的大致原理?

说一下你用的 Spring + Hibernate 的方框图? 您都用到了哪些部分?

请用英文简单介绍一下自己.

请把 <http://tomcat.apache.org/> 首页的这一段话用中文翻译一下?

Apache Tomcat is the servlet container that is used in the official Reference Implementation for the [Java Servlet](#) and [JavaServer Pages](#) technologies. The Java Servlet and JavaServer Pages specifications are developed by Sun under the [Java Community Process](#).

Apache Tomcat is developed in an open and participatory environment and released under the [Apache Software License](#). Apache Tomcat is intended to be a collaboration of the best-of-breed developers from around the world. We invite you to participate in this open development project. To learn more about getting involved, [click here](#).

Apache Tomcat powers numerous large-scale, mission-critical web applications across a diverse range of industries and organizations. Some of these users and their stories are listed on the [PoweredBy](#) wiki page.

```
try {  
    Connection conn = ...;  
    Statement stmt = ...;
```

```

ResultSet rs = stmt.executeQuery("select * from table1");

while(rs.next()) {

}
} catch(Exception ex) {
}

```

这段代码有什么不足之处?

HTML 的 form 提交之前如何验证数值不为空? 为空的话提示用户并终止提交?

为什么要用 ORM? 和 JDBC 有何不一样?

Weblogic 的 Server, Machine, Node, Domain 都有何区别?

简要讲一下 EJB 的 7 个 Transaction Level?

最常见的: 用 JDBC 查询学生成绩单, 把主要代码写出来.

Tomcat 5 用 query.jsp?name=中文, 怎么把这个变量无乱码的取出来?

[finally语句一定会执行么?](#)

前几天有一个朋友去面试, 被问到这样一个问题, 如下

```

1 public class smallT
2 {
3     public static void main(String args[])
4     {
5         smallT t = new smallT();
6         int b = t.get();
7         System.out.println(b);
8     }
9     public int get()
10    {
11        try
12        {
13            return 2 ;
14        }
15        catch (Exception e)

```

```

16  中... {
17  |return 3 ;
18  |      }
19  |finally
20  中... {
21  |return 4 ;
22  |      }
23  |      }
24  |}
25

```

输出:

4

我记得学习的时候说有返回值的方法, 执行完 return 语句后就会停止了; 另一方面我还记得在 try.....catch 语句中 finally 语句也是一定要执行的。于是在这里就产生了麻烦。试了一下, 发现果然是执行了 finally 中的值。正好这两天公司在培训, 我便把这个问题提给 java 很厉害的一个讲师, 他用断点测试了一下, 说是两个值都返回了.....

我又改了一小下, 在 get 方法中逐个添上了输出信息到控制台的语句, 发现居然能执行。如下

```

1  public class test
2  中... {
3  |  public static void main(String[] args)
4  中... {
5  |          test t = new test();
6  |  int b = t.get();
7  |          System.out.println("Third: a = "+b);
8  |      }
9  |  public int get()
10  中... {
11  |  try
12  中... {
13  |          System.out.println("First: a = "+2);
14  |  return 2;
15  |      }
16  |  catch(Exception e)
17  中... {

```

```

18 |         System.out.println(3);
19 |     }
20 | finally
21 | 白田... {
22 |         System.out.println("Second: a = "+4);
23 |     return 4;
24 |     }
25 |     }
26 | }
27

```

输出:

First: a = 2

Second: a = 4

Third: a = 4

可这明明是一个需要返回一个 int 整数的方法啊。

一道面试题目

一个整数，大于 0，不用循环和本地变量，按照 n, 2n, 4n, 8n 的顺序递增，当值大于 5000 时，把值按照指定顺序输出来。

例: n=1237

则输出为:

```

1237,
2474,
4948,
9896,
9896,
4948,
2474,
1237,

```

Tarena Wuhan

美资软件公司JAVA工程师电话面试题目

1. Talk about overriding, overloading.
2. Talk about JAVA design patterns you known.
3. Talk about the difference between LinkedList, ArrayList and Vector.
4. Talk about the difference between an Abstract class and an Interface.
5. Class a = new Class(); Class b = new Class();
if(a == b) returns true or false, why?
6. Why we use StringBuffer when concatenating strings?
7. Try to explain Singleton to us? Is it thread safe? If no, how to make it thread safe?
8. Try to explain Ioc?

9. How to set many-to-many relationship in Hibernate?
10. Talk about the difference between INNER JOIN and LEFT JOIN.
11. Why we use index in database? How many indexes is the maximum in one table as your suggestion?
12. When 'Final' is used in class, method and property, what does it mean?
13. Do you have any experience on XML? Talk about any XML tool you used, e.g. JAXB, JAXG.
14. Do you have any experience on Linux?
15. In OOD what is the reason when you create a Sequence diagram?

补一个内部培训用的 PPT: [SQL Tuning in Sybase.zip](#) 17KB, 英文, Sybase 调优, 大部分道理是通用的.

数据库三范式是什么? 别看问题简单, 我也经常答不上来被面试的人写上数据库不行.

还有就是 Sun Java 认证时候会考的一些英文选择题, 填空题, 模拟题也成.

[赴港JAVA开发工程师的面试题目](#)

也是上个星期五的上午, 按照与猎头的约定, 接受了香港某软件供应商的面试。工作是分析程序员, 需赴港工作, 以下是面试过程以及题目 (大概记下了 90%), 记录下来与大家分享。

第一部分: 例行公事的英文自我介绍;

以下部分必须以粤语回答 (本人非广东人, 粤语会讲, 但是不标准)。

第二部分: 项目经验介绍 (着重介绍项目背景, 开发流程以及本人在项目开发过程中的角色;

第三部分: 面试官根据简历提问:

- (1) 说出 Abstract class 与 interface 的不同?
- (2) Oracle 中如何进行错误处理? 如果用户反应速度慢, 你如何着手解决问题?
- (3) 图画板上列了两个 table, 问查询结果, 主要是考 inner join 与 left join 的。
- (4) union 和 union all 有什么不同?
- (5) 你用什么软件做设计?
- (6) 是否用过 Websphere?
- (7) iBatis 与 Hibernate 有什么不同?
- (8) 谈谈 Struts 中的 Action servlet。
- (9) 是否开发过 IBM portal 项目。
- (10) 是否介意加班?
- (11) 如果你去香港工作, 你认为你最大的困难是什么?

第四部分: 笔试, 三个英文考试题目选一个作答, 内容都是写一份 email。

出来后问了猎头, 他说最迟一个星期内就有结果。

结果，晚上就接到电话说通过了，并收到了合同和赴港申请的电子文件。工作地点是九龙，月薪 18K（中等偏低），合同期一年。猎头催我尽快签合同，然后他们马上去办工作签证。

犹豫 ing。

[每个JAVA初学者应该知道的问题](#)

对于这个系列里的问题，每个学 Java 的人都应该搞懂。当然，如果只是学 Java 玩玩就无所谓了。如果你认为自己已经超越初学者了，却不很懂这些问题，请将你自己重归初学者行列。内容均来自于 CSDN 的经典老贴。

问题一：我声明了什么！

```
String s = "Hello world!";
```

许多人都做过这样的事情，但是，我们到底声明了什么？回答通常是：一个 String，内容是“Hello world!”。这样模糊的回答通常是概念不清的根源。如果要准确的回答，一半的人大概会回答错误。

这个语句声明的是一个指向对象的引用，名为“s”，可以指向类型为 String 的任何对象，目前指向“Hello world!”这个 String 类型的对象。这就是真正发生的事情。我们并没有声明一个 String 对象，我们只是声明了一个只能指向 String 对象的引用变量。所以，如果在刚才那句语句后面，如果再运行一句：

```
String string = s;
```

我们是声明了另外一个只能指向 String 对象的引用，名为 string，并没有第二个对象产生，string 还是指向原来那个对象，也就是，和 s 指向同一个对象。

问题二：“==”和 equals 方法究竟有什么区别？

== 操作符专门用来比较变量的值是否相等。比较好理解的一点是：

```
int a=10;
```

```
int b=10;
```

则 a==b 将是 true。

但不好理解的地方是：

```
String a=new String("foo");
```

```
String b=new String("foo");
```

则 a==b 将返回 false。

根据前一帖说过，对象变量其实是一个引用，它们的值是指向对象所在的内存地址，而不是对象本身。a 和 b 都使用了 new 操作符，意味着将在内存中产生两个内容为“foo”的字符串，既然是“两个”，它们自然位于不同的内存地址。a 和 b 的值其实是两个不同的内存地址的值，所以使用“==”操作符，结果会是 false。诚然，a 和 b 所指的 object，它们的内容都是“foo”，应该是“相等”，但是 == 操作符并不涉及到对象内容的比较。

对象内容的比较，正是 equals 方法做的事。

看一下 Object 对象的 equals 方法是如何实现的：

```
boolean equals(Object o){
    return this==o;
}
```

Object 对象默认使用了 == 操作符。所以如果你自创的类没有覆盖 equals 方法，那你的类使用 equals 和使用 == 会得到同样的结果。同样也可以看出，Object

的 equals 方法没有达到 equals 方法应该达到的目标: 比较两个对象内容是否相等。因为答案应该由类的创建者决定, 所以 Object 把这个任务留给了类的创建者。

看一下一个极端的类:

```
Class Monster{
private String content;
...
boolean equals(Object another){ return true;}
}
```

我覆盖了 equals 方法。这个实现会导致无论 Monster 实例内容如何, 它们之间的比较永远返回 true。

所以当你是用 equals 方法判断对象的内容是否相等, 请不要想当然。因为可能你认为相等, 而这个类的作者不这样认为, 而类的 equals 方法的实现是由他掌握的。如果你需要使用 equals 方法, 或者使用任何基于散列码的集合

(HashSet, HashMap, HashTable), 请察看一下 java doc 以确认这个类的 equals 逻辑是如何实现的。

问题三: String 到底变了没有?

没有。因为 String 被设计成不可变(immutable)类, 所以它的所有对象都是不可变对象。请看下列代码:

```
String s = "Hello";
s = s + " world!";
```

s 所指向的对象是否改变了呢? 从本系列第一篇的结论很容易导出这个结论。我们来看看发生了什么事情。在这段代码中, s 原先指向一个 String 对象, 内容是 "Hello", 然后我们对 s 进行了+操作, 那么, s 所指向的那个对象是否发生了改变呢? 答案是没有。这时, s 不指向原来那个对象了, 它指向了另一个 String 对象, 内容为 "Hello world!", 原来那个对象还存在于内存之中, 只是 s 这个引用变量不再指向它了。

通过上面的说明, 我们很容易导出另一个结论, 如果经常对字符串进行各种各样的修改, 或者说, 不可见的修改, 那么使用 String 来代表字符串的话会引起很大的内存开销。因为 String 对象建立之后不能再改变, 所以对于每一个不同的字符串, 都需要一个 String 对象来表示。这时, 应该考虑使用 StringBuffer 类, 它允许修改, 而不是每个不同的字符串都要生成一个新的对象。并且, 这两种类型的对象转换十分容易。

同时, 我们还可以知道, 如果要使用内容相同的字符串, 不必每次都 new 一个 String。例如我们要在构造器中对一个名叫 s 的 String 引用变量进行初始化, 把它设置为初始值, 应当这样做:

```
public class Demo {
private String s;
...
public Demo {
s = "Initial Value";
}
...
}
```

而非

```
s = new String("Initial Value");
```

后者每次都会调用构造器，生成新对象，性能低下且内存开销大，并且没有意义，因为 String 对象不可改变，所以对于内容相同的字符串，只要一个 String 对象来表示就可以了。也就是说，多次调用上面的构造器创建多个对象，他们的 String 类型属性 s 都指向同一个对象。

上面的结论还基于这样一个事实：对于字符串常量，如果内容相同，Java 认为它们代表同一个 String 对象。而用关键字 new 调用构造器，总是会创建一个新的对象，无论内容是否相同。

至于为什么要把 String 类设计成不可变类，是它的用途决定的。其实不只 String，很多 Java 标准类库中的类都是不可变的。在开发一个系统的时候，我们有时候也需要设计不可变类，来传递一组相关的值，这也是面向对象思想的体现。不可变类有一些优点，比如因为它的对象是只读的，所以多线程并发访问也不会有任何问题。当然也有一些缺点，比如每个不同的状态都要一个对象来代表，可能会造成性能上的问题。所以 Java 标准类库还提供了一个可变版本，即 StringBuffer。

问题四：final 关键字到底修饰了什么？

final 使得被修饰的变量“不变”，但是由于对象型变量的本质是“引用”，使得“不变”也有了两种含义：引用本身的不变，和引用指向的对象不变。

引用本身的不变：

```
final StringBuffer a=new StringBuffer("immutable");
final StringBuffer b=new StringBuffer("not immutable");
a=b;//编译期错误
```

引用指向的对象不变：

```
final StringBuffer a=new StringBuffer("immutable");
a.append(" broken");//编译通过
```

可见，final 只对引用的“值”（也就是它所指向的那个对象的内存地址）有效，它迫使引用只能指向初始指向的那个对象，改变它的指向会导致编译期错误。至于它所指向的对象的变化，final 是不负责的。这很类似==操作符：==操作符只负责引用的“值”相等，至于这个地址所指向的对象内容是否相等，==操作符是不管的。

理解 final 问题有很重要的含义。许多程序漏洞都基于此——final 只能保证引用永远指向固定对象，不能保证那个对象的状态不变。在多线程的操作中，一个对象会被多个线程共享或修改，一个线程对对象无意识的修改可能会导致另一个使用此对象的线程崩溃。一个错误的解决方法就是在此对象新建的时候把它声明为 final，意图使得它“永远不变”。其实那是徒劳的。

问题五：到底要怎么样初始化！

本问题讨论变量的初始化，所以先来看一下 Java 中有哪些种类的变量。

1. 类的属性，或者叫值域
2. 方法里的局部变量
3. 方法的参数

对于第一种变量，Java 虚拟机会自动进行初始化。如果给出了初始值，则初始化为该初始值。如果没有给出，则把它初始化为该类型变量的默认初始值。

int 类型变量默认初始值为 0

float 类型变量默认初始值为 0.0f

double 类型变量默认初始值为 0.0

boolean 类型变量默认初始值为 false

char 类型变量默认初始值为 0(ASCII 码)

long 类型变量默认初始值为 0

所有对象引用类型变量默认初始值为 null，即不指向任何对象。注意数组本身也是对象，所以没有初始化的数组引用在自动初始化后其值也是 null。

对于两种不同的类属性，static 属性与 instance 属性，初始化的时机是不同的。instance 属性在创建实例的时候初始化，static 属性在类加载，也就是第一次用到这个类的时候初始化，对于后来的实例的创建，不再次进行初始化。这个问题会在以后的系列中进行详细讨论。

对于第二种变量，必须明确地进行初始化。如果再没有初始化之前就试图使用它，编译器会抗议。如果初始化的语句在 try 块中或 if 块中，也必须要让它在第一次使用前一定能够得到赋值。也就是说，把初始化语句放在只有 if 块的条件判断语句中编译器也会抗议，因为执行的时候可能不符合 if 后面的判断条件，如此一来初始化语句就不会被执行了，这就违反了局部变量使用前必须初始化的规定。但如果在 else 块中也有初始化语句，就可以通过编译，因为无论如何，总有至少一条初始化语句会被执行，不会发生使用前未被初始化的事情。对于 try-catch 也是一样，如果只有在 try 块里才有初始化语句，编译部通过。如果在 catch 或 finally 里也有，则可以通过编译。总之，要保证局部变量在使用之前一定被初始化了。所以，一个好的做法是在声明他们的时候就初始化他们，如果不知道要出事化成什么值好，就用上面的默认值吧！

其实第三种变量和第二种本质上是一样的，都是方法中的局部变量。只不过作为参数，肯定是被初始化过的，传入的值就是初始值，所以不需要初始化。

问题六：instanceof 是什么东东？

instanceof 是 Java 的一个二元操作符，和 ==、>、< 是同一类东东。由于它是由字母组成的，所以也是 Java 的保留关键字。它的作用是测试它左边的对象是否是它右边的类的实例，返回 boolean 类型的数据。举个例子：

```
String s = "I AM an Object!";
```

```
boolean isObject = s instanceof Object;
```

我们声明了一个 String 对象引用，指向一个 String 对象，然后用 instanceof 来测试它所指向的对象是否是 Object 类的一个实例，显然，这是真的，所以返回 true，也就是 isObject 的值为 True。

instanceof 有一些用处。比如我们写了一个处理账单的系统，其中有这样三个类：

```
public class Bill { //省略细节 }
```

```
public class PhoneBill extends Bill { //省略细节 }
```

```
public class GasBill extends Bill { //省略细节 }
```

在处理程序里有一个方法，接受一个 Bill 类型的对象，计算金额。假设两种账单计算方法不同，而传入的 Bill 对象可能是两种中的任何一种，所以要用

instanceof 来判断：

```
public double calculate(Bill bill) {
```

```
    if (bill instanceof PhoneBill) {
```

```
        //计算电话账单
```

```

}
if (bill instanceof GasBill) {
//计算燃气账单
}
...
}

```

这样就可以用一个方法处理两种子类。

然而，这种做法通常被认为是没有好好利用面向对象中的多态性。其实上面的功能要求用方法重载完全可以实现，这是面向对象变成应有的做法，避免回到结构化编程模式。只要提供两个名字和返回值都相同，接受参数类型不同的方法就可以了：

```

public double calculate(PhoneBill bill) {
//计算电话账单
}
public double calculate(GasBill bill) {
//计算燃气账单
}

```

所以，使用 instanceof 在绝大多数情况下并不是推荐的做法，应当好好利用多态。

BeanSoft 参加过的一次面试：

2004 年 8 月 18 日 星期三 【农历 甲申 猴年 七月初三】

笔试题目

1. 写出常用的 Linux 命令
 - a) 列出当前目录
 - b) 列出所有系统变量
 - c) 重命名文件
 - d) etc... 记不住了.

2. Write a Java application use the Singleton pattern, it should have one instance per class. It should have two integer variable, one is count, one is initia. There should be two methods in this class, and the method should be thread safely in multi-thread environment.

- a) a count() method, after each call the count should be added on by 1;
- b) a reset() method, after each call the count should be set to the value of initia.

3. Write a JavaBean and a JSP file. The page should output a date string in this pattern "今天是 2004 年 8 月 15 日上午 10:00", the value is taken from the bean.

4. 以下三个题目，任选其一或多个：

- 1) 忘了...，是关于 Java 的.
- 2) 写一个应用程序，读出 STUDENT 表中的数据并打印出所有名称. 数据库系统任选.
- 3) 写一个 Servlet，读取名为 url 的参数，并连接到此字符串指定的地址上，读取所有内容后显示给客户.

5. 6. 7. 都是关于 SQL 的，例如 SELECT, UPDATE, DELETE 之类的，还有的有子查询. 数据库系统没有限制.

8. Write a html file, it should have the following functions:

- 1) check whether the user's name is empty;
- 2) the email address should have a '@';
- 3) telephone number must be '1'-'9', '-', ' ' (space).

Page picture:

Please input your name:

[] [Check input]

Please input your address:

[abc@] [Check input]

Please input your telephone number:

[123a] [Check input]

Java 面试题及其答案

前段时间因为要参加一个笔试，在准备期间在网上找到了两条关于笔试题目的文章，其中一篇为《[注意：应聘 Java 笔试时可能出现的面试题](#)》，还有一篇忘了名字，读后深受启发。

在寻找这些答案的过程中，我将相关答案记录下来，就形成了以下这些东西。需要说明的是以下答案肯定有很多不完整甚至错误的地方，需要各位来更正与完善它，千万不要扔我的鸡蛋啊。

希望本文能够给即将奔赴笔试考场的同仁些许帮助，更希望更多的人加入到收集整理笔试题与完善答案的这些工作中来，为大家更好的获得工作机会做一点贡献。

在此感谢前面两文的作者的对笔试题目的收集与整理。

如有任何意见与建议请通过 QQ:6045306, Mail:huijunzi@21cn.com 与我联系。

Java 基础方面：

1、作用域 public, private, protected, 以及不写时的区别

答：区别如下：

作用域 当前类 同一 package 子孙类 其他 package

public ✓ ✓ ✓ ✓

protected ✓ ✓ ✓ ×

friendly ✓ ✓ × ×

private ✓ × × ×

不写时默认为 friendly

2、ArrayList 和 Vector 的区别,HashMap 和 Hashtable 的区别

答:就 ArrayList 与 Vector 主要从二方面来说.

一.同步性:Vector 是线程安全的,也就是说是同步的,而 ArrayList 是线程程序不安全的,不是同步的

二.数据增长:当需要增长时,Vector 默认增长为原来一倍,而 ArrayList 却是原来的一半

就 HashMap 与 Hashtable 主要从三方面来说.

一.历史原因:Hashtable 是基于陈旧的 Dictionary 类的,HashMap 是 Java 1.2 引进的 Map 接口的一个实现

二.同步性:Hashtable 是线程安全的,也就是说是同步的,而 HashMap 是线程程序不安全的,不是同步的

三.值:只有 HashMap 可以让你将空值作为一个表的条目的 key 或 value

3、char 型变量中能不能存贮一个中文汉字?为什么?

答:是能够定义成为一个中文的,因为 java 中以 unicode 编码,一个 char 占 16 个字节,所以放一个中文是没问题的

4、多线程有几种实现方法,都是什么?同步有几种实现方法,都是什么?

答:多线程有两种实现方法,分别是继承 Thread 类与实现 Runnable 接口
同步的实现方面有两种,分别是 synchronized,wait 与 notify

5、继承时候类的执行顺序问题,一般都是选择题,问你将会打印出什么?

答:父类:

```
package test;
public class FatherClass
{
    public FatherClass()
    {
        System.out.println("FatherClass Create");
    }
}
```

子类:

```
package test;
import test.FatherClass;
public class ChildClass extends FatherClass
{
    public ChildClass()
    {
        System.out.println("ChildClass Create");
    }
    public static void main(String[] args)
    {
        FatherClass fc = new FatherClass();
        ChildClass cc = new ChildClass();
    }
}
```

```
}
```

输出结果:

```
C:\>java test.ChildClass
FatherClass Create
FatherClass Create
ChildClass Create
```

6、内部类的实现方式?

答: 示例代码如下:

```
package test;
public class OuterClass
{
    private class InterClass
    {
        public InterClass()
        {
            System.out.println("InterClass Create");
        }
    }
    public OuterClass()
    {
        InterClass ic = new InterClass();
        System.out.println("OuterClass Create");
    }
    public static void main(String[] args)
    {
        OuterClass oc = new OuterClass();
    }
}
```

输出结果:

```
C:\>java test/OuterClass
InterClass Create
OuterClass Create
```

再一个例题:

```
public class OuterClass {
    private double d1 = 1.0;
    //insert code here
}
```

You need to insert an inner class declaration at line 3. Which two inner class declarations are

valid?(Choose two.)

```
A. class InnerOne{
    public static double methoda() {return d1;}
```

```

}
B. public class InnerOne{
static double methoda() {return d1;}
}
C. private class InnerOne{
double methoda() {return d1;}
}
D. static class InnerOne{
protected double methoda() {return d1;}
}
E. abstract class InnerOne{
public abstract double methoda();
}

```

说明如下：

- 一. 静态内部类可以有静态成员，而非静态内部类则不能有静态成员。故 A、B 错
- 二. 静态内部类的非静态成员可以访问外部类的静态变量，而不可访问外部类的非静态变量；return d1 出错。

故 D 错

三. 非静态内部类的非静态成员可以访问外部类的非静态变量。 故 C 正确

四. 答案为 C、E

7、垃圾回收机制, 如何优化程序?

希望大家补上, 谢谢

8、float 型 float 3.4 是否正确?

答: 不正确。精度不准确, 应该用强制类型转换, 如下所示: float f=(float)3.4

9、介绍 JAVA 中的 Collection Framework (包括如何写自己的数据结构)?

答: Collection Framework 如下:

```

Collection
├─List
│   ├─LinkedList
│   ├─ArrayList
│   └─Vector
│       └─Stack
└─Set
Map
├─Hashtable
├─HashMap
└─WeakHashMap

```

Collection 是最基本的集合接口, 一个 Collection 代表一组 Object, 即 Collection 的元素 (Elements)

Map 提供 key 到 value 的映射

10、Java 中异常处理机制，事件机制？

11、JAVA 中的多形与继承？

希望大家补上，谢谢

12、抽象类与接口？

答：抽象类与接口都用于抽象，但是抽象类(JAVA 中)可以有自己的部分实现，而接口则完全是一个标识(同时有多重继承的功能)。

13、Java 的通信编程，编程题(或问答)，用 JAVA SOCKET 编程，读服务器几个字符，再写入本地显示？

答:Server 端程序:

```
package test;
import java.net.*;
import java.io.*;

public class Server
{
    private ServerSocket ss;
    private Socket socket;
    private BufferedReader in;
    private PrintWriter out;
    public Server()
    {
        try
        {
            ss=new ServerSocket(8080);
            while(true)
            {
                socket = ss.accept();
                String RemoteIP = socket.getInetAddress().getHostAddress();
                String RemotePort = ":"+socket.getLocalPort();
                System.out.println("A client come in!IP:"+RemoteIP+RemotePort);
                in = new BufferedReader(new
                    InputStreamReader(socket.getInputStream()));
                String line = in.readLine();
                System.out.println("Cleint send is :"+ line);
                out = new PrintWriter(socket.getOutputStream(), true);
                out.println("Your Message Received!");
                out.close();
                in.close();
                socket.close();
            }
        } catch (IOException e)
```

```
{
out.println("wrong");
}
}
public static void main(String[] args)
{
new Server();
}
};
Client 端程序:
package test;
import java.io.*;
import java.net.*;

public class Client
{
Socket socket;
BufferedReader in;
PrintWriter out;
public Client()
{
try
{
System.out.println("Try to Connect to 127.0.0.1:8000");
socket = new Socket("127.0.0.1",8000);
System.out.println("The Server Connected");
System.out.println("Please enter some Character:");
BufferedReader line = new BufferedReader(new
InputStreamReader(System.in));
out = new PrintWriter(socket.getOutputStream(),true);
out.println(line.readLine());
in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
System.out.println(in.readLine());
out.close();
in.close();
socket.close();
}catch(IOException e)
{
out.println("Wrong");
}
}
}
public static void main(String[] args)
{
```

```

new Client();
}
};

```

14、用 JAVA 实现一种排序, JAVA 类实现序列化的方法(二种)? 如在 COLLECTION 框架中, 实现比较要实现什么样的接口?

答:用插入法进行排序代码如下

```

package test;
import java.util.*;
class InsertSort
{
    ArrayList al;
    public InsertSort(int num,int mod)
    {
        al = new ArrayList(num);
        Random rand = new Random();
        System.out.println("The ArrayList Sort Before:");
        for (int i=0;i<num ;i++ )
        {
            al.add(new Integer(Math.abs(rand.nextInt()) % mod + 1));
            System.out.println("al["+i+"]="+al.get(i));
        }
    }
    public void SortIt()
    {
        Integer tempInt;
        int MaxSize=1;
        for(int i=1;i<al.size();i++)
        {
            tempInt = (Integer)al.remove(i);
            if(tempInt.intValue() >= ((Integer)al.get(MaxSize-1)).intValue())
            {
                al.add(MaxSize, tempInt);
                MaxSize++;
                System.out.println(al.toString());
            } else {
                for (int j=0;j<MaxSize ;j++ )
                {
                    if
                    (((Integer)al.get(j)).intValue() >= tempInt.intValue())
                    {
                        al.add(j, tempInt);
                        MaxSize++;
                    }
                }
            }
        }
    }
}

```

```

System.out.println(al.toString());
break;
}
}
}
}
System.out.println("The ArrayList Sort After:");
for(int i=0;i<al.size();i++)
{
System.out.println("al["+i+"]="+al.get(i));
}
}
public static void main(String[] args)
{
InsertSort is = new InsertSort(10,100);
is.SortIt();
}
}

```

JAVA 类实现序列化化的方法是实现 java.io.Serializable 接口

Collection 框架中实现比较要实现 Comparable 接口和 Comparator 接口

15、编程：编写一个截取字符串的函数，输入为一个字符串和字节数，输出为按字节截取的字符串。但是要保证汉字不被截半个，如“我ABC”4，应该截为“我AB”，输入“我ABC 汉 DEF”，6，应该输出为“我ABC”而不是“我ABC+汉的半个”。

答：代码如下：

```

package test;

class SplitString
{
String SplitStr;
int SplitByte;
public SplitString(String str,int bytes)
{
SplitStr=str;
SplitByte=bytes;
System.out.println("The String
is:' "+SplitStr+"' SplitBytes="+SplitByte);
}
public void SplitIt()
{
int loopCount;

loopCount=(SplitStr.length()%SplitByte==0)?(SplitStr.length()/SplitByte):(SplitStr.length()/Split

```

```

Byte+1);
System.out.println("Will Split into "+loopCount);
for (int i=1;i<=loopCount ;i++ )
{
if (i==loopCount){

System.out.println(SplitStr.substring((i-1)*SplitByte, SplitStr.length
()));
} else {

System.out.println(SplitStr.substring((i-1)*SplitByte, (i*SplitByte)));
}
}
}
public static void main(String[] args)
{
SplitString ss = new SplitString("test 中 dd 文 dsaf 中男大 3443n 中国 43
中国人

0ewldfls=103",4);
ss.SplitIt();
}
}

```

16、JAVA 多线程编程。 用 JAVA 写一个多线程程序，交互四个线程，二个加 1，二个对一个变量减一，输出，希望大家补上，谢谢

17、STRING 与 STRINGBUFFER 的区别。

答：STRING 的长度是不可变的，STRINGBUFFER 的长度是可变的。如果你对字符串中的内容经常进行操作，特别是内容要修改时，那么使用 StringBuffer，如果最后需要 String，那么使用 StringBuffer 的 toString() 方法

Jsp 方面

1、jsp 有哪些内置对象?作用分别是什么?

答:JSP 共有以下 9 种基本内置组件（可与 ASP 的 6 种内部组件相对应）:

- request 用户端请求，此请求会包含来自 GET/POST 请求的参数
- response 网页传回用户端的回应
- pageContext 网页的属性是在这里管理
- session 与请求有关的会话期
- application servlet 正在执行的内容
- out 用来传送回应的输出
- config servlet 的构架部件

page JSP 网页本身

exception 针对错误网页，未捕捉的例外

2、jsp 有哪些动作?作用分别是什么?

答:JSP 共有以下 6 种基本动作

jsp:include: 在页面被请求的时候引入一个文件。

jsp:useBean: 寻找或者实例化一个 JavaBean。

jsp:setProperty: 设置 JavaBean 的属性。

jsp:getProperty: 输出某个 JavaBean 的属性。

jsp:forward: 把请求转到一个新的页面。

jsp:plugin: 根据浏览器类型为 Java 插件生成 OBJECT 或 EMBED 标记

3、JSP 中动态 INCLUDE 与静态 INCLUDE 的区别?

答: 动态 INCLUDE 用 jsp:include 动作实现

<jsp:include page="included.jsp" flush="true" />它总是会检查所含文件中的变化, 适合用于包含动态页面, 并且可以带参数

静态 INCLUDE 用 include 伪码实现, 定不会检查所含文件的变化, 适用于包含静态页面

<%@ include file="included.htm" %>

4、两种跳转方式分别是什么?有什么区别?

答: 有两种, 分别为:

<jsp:include page="included.jsp" flush="true"

<jsp:forward page="nextpage.jsp"/>

前者页面不会转向 include 所指的页面, 只是显示该页的结果, 主页面还是原来的页面。执行完后还会回来, 相当于函数调用。并且可以带参数。后者完全转向新页面, 不会再回来。相当于 go to 语句。

Servlet 方面

1、说一说 Servlet 的生命周期?

答: servlet 有良好的生存期的定义, 包括加载和实例化、初始化、处理请求以及服务结束。这个生存期由 javax.servlet.Servlet 接口的 init, service 和 destroy 方法表达。

2、Servlet 版本间(忘了问的是哪两个版本了)的不同?

希望大家补上, 谢谢

3、JAVA SERVLET API 中 forward() 与 redirect() 的区别?

答: 前者仅是容器中控制权的转向, 在客户端浏览器地址栏中不会显示出转向后的地址; 后者则是完全的跳转, 浏览器将会得到跳转的地址, 并重新发送请求链接。这样, 从浏览器的地址栏中可以看到跳转后的链接地址。所以, 前者更加高效, 在前者可以满足需要时, 尽量使用 forward() 方法, 并且, 这样也有助于隐藏实际的链接。在有些情况下, 比如, 需要跳转到一个其它服务器上的资源, 则必须使用 sendRedirect() 方法。

4、Servlet 的基本架构

```
public class ServletName extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse
    response) throws
    ServletException, IOException {
    }
    public void doGet(HttpServletRequest request, HttpServletResponse
    response) throws
    ServletException, IOException {
    }
}
```

Jdbc、Jdo 方面

1、可能会让你写一段 Jdbc 连 Oracle 的程序, 并实现数据查询.

答:程序如下:

```
package hello.ant;
import java.sql.*;
public class jdbc
{
    String dbUrl="jdbc:oracle:thin:@127.0.0.1:1521:orcl"
    String theUser="admin"
    String thePw="manager"
    Connection c=null;
    Statement conn;
    ResultSet rs=null;
    public jdbc()
    {
        try{
            Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();
            c = DriverManager.getConnection(dbUrl, theUser, thePw);
            conn=c.createStatement();
        }catch(Exception e){
            e.printStackTrace();
        }
    }
    public boolean executeUpdate(String sql)
    {
        try
        {
            conn.executeUpdate(sql);
            return true;
        }
        catch (SQLException e)
```

```
{
e.printStackTrace();
return false;
}
}
public ResultSet executeQuery(String sql)
{
rs=null;
try
{
rs=conn.executeQuery(sql);
}
catch (SQLException e)
{
e.printStackTrace();
}
return rs;
}
public void close()
{
try
{
conn.close();
c.close();
}
catch (Exception e)
{
e.printStackTrace();
}
}
public static void main(String[] args)
{
ResultSet rs;
jdbc conn = new jdbc();
rs=conn.executeQuery("select * from test");
try{
while (rs.next())
{
System.out.println(rs.getString("id"));
System.out.println(rs.getString("name"));
}
} catch (Exception e)
{
e.printStackTrace();
}
```

```

}
}
}

```

2、Class.forName 的作用?为什么要用?

答: 调用该访问返回一个以字符串指定类名的类的对象。

3、Jdo 是什么?

答:JDO 是 Java 对象持久化的新的规范, 为 java data object 的简称, 也是一个用于存取某种数据仓库中的对象的标准 API。JDO 提供了透明的对象存储, 因此对开发人员来说, 存储数据对象完全不需要额外的代码(如 JDBC API 的使用)。这些繁琐的例行工作已经转移到 JDO 产品提供商身上, 使开发人员解脱出来, 从而集中时间和精力在业务逻辑上。另外, JDO 很灵活, 因为它可以在任何数据底层上运行。JDBC 只是面向关系数据库 (RDBMS) JDO 更通用, 提供到任何数据底层的存储功能, 比如关系数据库、文件、XML 以及对象数据库 (ODBMS) 等等, 使得应用可移植性更强。

4、在 ORACLE 大数据量下的分页解决方法。一般用截取 ID 方法, 还有是三层嵌套方法。

答:一种分页方法

```

<%
int i=1;
int numPages=14;
String pages = request.getParameter("page");
int currentPage = 1;
currentPage=(pages==null)?Integer.parseInt("1"):Integer.parseInt(pages);
sql = "select count(*) from tables";
ResultSet rs = DBLink.executeQuery(sql);
while(rs.next()) i = rs.getInt(1);
int intPageCount=i;
intPageCount=(i%numPages==0)?(i/numPages):(i/numPages+1);
int nextPage;
int upPage;
nextPage = currentPage+1;
if (nextPage>intPageCount) nextPage=intPageCount;
upPage = currentPage-1;
if (upPage<=1) upPage=1;
rs.close();
sql="select * from tables"
rs=DBLink.executeQuery(sql);
i=0;
while((i<numPages*(currentPage-1))&&rs.next()) {i++;}
%>
//输出内容
//输出翻页连接

```

```

合计:<%=currentPage%>/<%=intPageCount%><a href="List.jsp?page=1"
temp_href="List.jsp?page=1">第一页</a><a

href="List.jsp?page=<%=upPage%>"
temp_href="List.jsp?page=<%=upPage%>">上一页</a>
<%
for(int j=1;j<=intPageCount;j++){
if(currentPage!=j){
%>
<a href="list.jsp?page=<%=j%>"
temp_href="list.jsp?page=<%=j%>">[<%=j%>]</a>
<%
} else {
out.println(j);
}
}
%>
<a href="List.jsp?page=<%=nextPage%>"
temp_href="List.jsp?page=<%=nextPage%>">下一页</a><a
href="List.jsp?page=<%=intPageCount%>"
temp_href="List.jsp?page=<%=intPageCount%>">最后页

</a>

```

Xml 方面

1、xml 有哪些解析技术?区别是什么?

答:有 DOM, SAX, STAX 等

DOM:处理大型文件时其性能下降的非常厉害。这个问题是由 DOM 的树结构所造成的,这种结构占用的内存较多,而且 DOM 必须在解析文件之前把整个文档装入内存,适合对 XML 的随机访问 SAX:不现于 DOM, SAX 是事件驱动型的 XML 解析方式。它顺序读取 XML 文件,不需要一次全部装载整个文件。当遇到像文件开头,文档结束,或者标签开头与标签结束时,它会触发一个事件,用户通过在其回调事件中写入处理代码来处理 XML 文件,适合对 XML 的顺序访问

STAX:Streaming API for XML (StAX)

2、你在项目中用到了 xml 技术的哪些方面?如何实现的?

答:用到了数据存贮,信息配置两方面。在做数据交换平台时,将不能数据源的数据组装成 XML 文件,然后将 XML 文件压缩打包加密后通过网络传送给接收者,接收解密与解压缩后再同 XML 文件中还原相关信息进行处理。在做软件配置时,利用 XML 可以很方便的进行,软件的各种配置参数都存贮在 XML 文件中。

3、用 jdom 解析 xml 文件时如何解决中文问题?如何解析?

答:看如下代码,用编码方式加以解决

```
package test;
```

```

import java.io.*;
public class DOMTest
{
    private String inFile = "c:\\people.xml"
    private String outFile = "c:\\people.xml"
    public static void main(String args[])
    {
        new DOMTest();
    }
    public DOMTest()
    {
        try
        {
            javax.xml.parsers.DocumentBuilder builder =
                javax.xml.parsers.DocumentBuilderFactory.newInstance().newDocumentBuilder();
            org.w3c.dom.Document doc = builder.newDocument();
            org.w3c.dom.Element root = doc.createElement("老师");
            org.w3c.dom.Element wang = doc.createElement("王");
            org.w3c.dom.Element liu = doc.createElement("刘");
            wang.appendChild(doc.createTextNode("我是王老师"));
            root.appendChild(wang);
            doc.appendChild(root);
            javax.xml.transform.Transformer transformer =
                javax.xml.transform.TransformerFactory.newInstance().newTransformer();
            transformer.setOutputProperty(javax.xml.transform.OutputKeys.ENCODING,
                "gb2312");
            transformer.setOutputProperty(javax.xml.transform.OutputKeys.INDENT,
                "yes");

            transformer.transform(new javax.xml.transform.dom.DOMSource(doc),
                new
                    javax.xml.transform.stream.StreamResult(outFile));
        }
        catch (Exception e)
        {
            System.out.println (e.getMessage());
        }
    }
}

```

4、编程用 JAVA 解析 XML 的方式.

答:用 SAX 方式解析 XML, XML 文件如下:

```
<?xml version="1.0" encoding="gb2312"?>
<person>
<name>王小明</name>
<college>信息学院</college>
<telephone>6258113</telephone>
<notes>男, 1955 年生, 博士, 95 年调入海南大学</notes>
</person>
```

事件回调类 SAXHandler. java

```
import java.io.*;
import java.util.Hashtable;
import org.xml.sax.*;

public class SAXHandler extends HandlerBase
{
private Hashtable table = new Hashtable();
private String currentElement = null;
private String currentValue = null;
public void setTable(Hashtable table)
{
this.table = table;
}
public Hashtable getTable()
{
return table;
}
public void startElement(String tag, AttributeList attrs)
throws SAXException
{
currentElement = tag;
}
public void characters(char[] ch, int start, int length)
throws SAXException
{
currentValue = new String(ch, start, length);
}
public void endElement(String name) throws SAXException
{
if (currentElement.equals(name))
table.put(currentElement, currentValue);
}
}
```

JSP 内容显示源码, SaxXml. jsp:

```
<HTML>
<HEAD>
<TITLE>剖析 XML 文件 people.xml</TITLE>
```

```

</HEAD>
<BODY>
<%@ page errorPage="ErrPage.jsp"
contentType="text/html; charset=GB2312" %>
<%@ page import="java.io.*" %>
<%@ page import="java.util.Hashtable" %>
<%@ page import="org.w3c.dom.*" %>
<%@ page import="org.xml.sax.*" %>
<%@ page import="javax.xml.parsers.SAXParserFactory" %>
<%@ page import="javax.xml.parsers.SAXParser" %>
<%@ page import="SAXHandler" %>
<%
File file = new File("c:\\people.xml");
FileReader reader = new FileReader(file);
Parser parser;
SAXParserFactory spf = SAXParserFactory.newInstance();
SAXParser sp = spf.newSAXParser();
SAXHandler handler = new SAXHandler();
sp.parse(new InputSource(reader), handler);
Hashtable hashTable = handler.getTable();
out.println("<TABLE BORDER=2><CAPTION>教师信息表</CAPTION>");
out.println("<TR><TD>姓名</TD>" + "<TD>" +
(String)hashTable.get(new String("name")) + "</TD></TR>");
out.println("<TR><TD>学院</TD>" + "<TD>" +
(String)hashTable.get(new String("college")) + "</TD></TR>");
out.println("<TR><TD>电话</TD>" + "<TD>" +
(String)hashTable.get(new String("telephone")) + "</TD></TR>");
out.println("<TR><TD>备注</TD>" + "<TD>" +
(String)hashTable.get(new String("notes")) + "</TD></TR>");
out.println("</TABLE>");
%>
</BODY>
</HTML>

```

EJB 方面

1、EJB2.0 有哪些内容?分别用在什么场合? EJB2.0 和 EJB1.1 的区别?

答:规范内容包括 Bean 提供者,应用程序装配者,EJB 容器,EJB 配置工具,EJB 服务提供者,系统管理员。这里面,EJB 容器是 EJB 之所以能够运行的核心。EJB 容器管理着 EJB 的创建,撤消,激活,去活,与数据库的连接等等重要的核心工作。JSP,Servlet,EJB,JNDI,JDBC,JMS.....

2、EJB 与 JAVA BEAN 的区别?

答:Java Bean 是可复用的组件,对 Java Bean 并没有严格的规范,理论上讲,任何一个 Java 类都可以是一个 Bean。但通常情况下,由于 Java Bean 是被容器

所创建（如 Tomcat）的，所以 Java Bean 应具有一个无参的构造器，另外，通常 Java Bean 还要实现 Serializable 接口用于实现 Bean 的持久性。Java Bean 实际上相当于微软 COM 模型中的本地进程内 COM 组件，它是不能被跨进程访问的。Enterprise Java Bean 相当于 DCOM，即分布式组件。它是基于 Java 的远程方法调用（RMI）技术的，所以 EJB 可以被远程访问（跨进程、跨计算机）。但 EJB 必须被布署在诸如 Webspere、WebLogic 这样的容器中，EJB 客户从不直接访问真正的 EJB 组件，而是通过其容器访问。EJB 容器是 EJB 组件的代理，EJB 组件由容器所创建和管理。客户通过容器来访问真正的 EJB 组件。

3、EJB 的基本架构

答：一个 EJB 包括三个部分：

Remote Interface 接口的代码

```
package Beans;
import javax.ejb.EJBObject;
import java.rmi.RemoteException;
public interface Add extends EJBObject
{
//some method declare
}
```

Home Interface 接口的代码

```
package Beans;
import java.rmi.RemoteException;
import javax.ejb.CreateException;
import javax.ejb.EJBHome;
public interface AddHome extends EJBHome
{
//some method declare
}
```

EJB 类的代码

```
package Beans;
import java.rmi.RemoteException;
import javax.ejb.SessionBean;
import javax.ejb.SessionContext;
public class AddBean implements SessionBean
{
//some method declare
}
```

J2EE, MVC 方面

1、MVC 的各个部分都有那些技术来实现?如何实现?

答：MVC 是 Model—View—Controller 的简写。“Model”代表的是应用的业务逻辑（通过 JavaBean, EJB 组件实现），“View”是应用的表示面（由 JSP 页面产生），“Controller”是提供应用的处理过程控制（一般是一个 Servlet），通

过这种设计模型把应用逻辑, 处理过程和显示逻辑分成不同的组件实现。这些组件可以进行交互和重用。

2、应用服务器与 WEB SERVER 的区别?

希望大家补上, 谢谢

3、J2EE 是什么?

答: J2EE 是 Sun 公司提出的多层(multi-tiered), 分布式(distributed), 基于组件(component-based)的企业级应用模型(enterprise application model). 在这样的一个应用系统中, 可按照功能划分为不同的组件, 这些组件又可在不同计算机上, 并且处于相应的层次(tier)中。所属层次包括客户层(client tier)组件, web 层和组件, Business 层和组件, 企业信息系统(EIS)层。

4、WEB SERVICE 名词解释。JSDDL 开发包的介绍。JAXP、JAXM 的解释。SOAP、UDDI, WSDL 解释。

答: Web Service 描述语言 WSDL

SOAP 即简单对象访问协议(Simple Object Access Protocol), 它是用于交换 XML 编码信息的轻量级协议。

UDDI 的目的是为电子商务建立标准; UDDI 是一套基于 Web 的、分布式的、为 Web Service 提供的、信息注册中心的实现标准规范, 同时也包含一组使企业能将自身提供的 Web Service 注册, 以使别的企业能够发现的访问协议的实现标准。

5、BS 与 CS 的联系与区别。

希望大家补上, 谢谢

6、STRUTS 的应用(如 STRUTS 结构)

答: Struts 是采用 Java Servlet/JSP/JavaServer Pages 技术, 开发 Web 应用程序的开源的 framework。采用 Struts 能开发出基于

MVC(Model-View-Controller)设计模式的应用构架。 Struts 有如下的主要功能:

- 一. 包含一个 controller servlet, 能将用户的请求发送到相应的 Action 对象。
- 二. JSP 自由 tag 库, 并且在 controller servlet 中提供关联支持, 帮助开发人员创建交互式表单应用。
- 三. 提供了一系列实用对象: XML 处理、通过 Java reflection APIs 自动处理 JavaBeans 属性、国际化的提示和消息。

设计模式方面

1、开发中都用到了那些设计模式?用在什么场合?

答: 每个模式都描述了一个在我们的环境中不断出现的问题, 然后描述了该问题的解决方案的核心。通过这种方式, 你可以无数次地使用那些已有的解决方案, 无需在重复相同的工作。主要用到了 MVC 的设计模式。用来开发 JSP/Servlet 或者 J2EE 的相关应用。简单工厂模式等。

2、UML 方面

答：标准建模语言 UML。用例图, 静态图(包括类图、对象图和包图), 行为图, 交互图(顺序图, 合作图), 实现图,

JavaScript 方面

1、如何校验数字型?

```
var re=/^\d{1,8}$|\. \d{1,2}$/;
var str=document.form1.all(i).value;
var r=str.match(re);
if (r==null)
{
    sign=-4;
    break;
}
else{
    document.form1.all(i).value=parseFloat(str);
}
```

CORBA 方面

1、CORBA 是什么?用途是什么?

答：CORBA 标准是公共对象请求代理结构(Common Object Request Broker Architecture)，由对象管理组织 (Object Management Group, 缩写为 OMG) 标准化。它的组成是接口定义语言 (IDL)、语言绑定 (Language Binding, 也译为联编) 和允许应用程序间互操作的协议。其目的为用不同的程序设计语言编写的程序在不同的进程中运行于不同的操作系统开发

LINUX 方面

1、Linux 下线程，GDI 类的解释。

答：Linux 实现的就是基于核心轻量级进程的“一对一”线程模型，一个线程实体对应一个核心轻量级进程，而线程之间的管理在核外函数库中实现。

GDI 类为图像设备编程接口类库。

Hibernate: 简述 Hibernate 和 JDBC 的优缺点? 如何书写一个 one to many 配置文件.

Spring 的依赖注入是什么意思? 给一个 Bean 的 message 属性, 字符串类型, 注入值为 "Hello" 的 XML 配置文件该怎么写?

再转载： 阅读完此贴你才可以大胆的高薪工作！

<http://bbs.tarena.com.cn/topic.cgi?forum=1&topic=10848&show=0>

大家找工作就要面试，有面试就有对付面试的办法。以下一些题目来自我朋友痛苦的面试经历，他给我总结了一下，我拿来和大家分享！提这些问题的公司包括IBM, E*Trade, Siebel, Motorola, SUN, 以及其它大小公司。

面试是没什么道理可讲的，它的题目有的不合情理、脱离实际。有在纸上写的，有当面考你的，也有在电话里问的，给你IDE的估计很少(否则你赶快去买彩票，说不定中)。所以如果你看完此文后，请不要抱怨说这些问题都能用IDE来解决。你必须在任何情况下准确回答这些问题，在面试中如果出现一两题回答不准确很有可能你就会被拒之门外了。当然这些都是Java的基本题，那些面试的人大多数不会问你Hibernate有多先进，Eclipse的三个组成部分，或command design pattern，他们都是老一辈了，最喜欢问的就是基础知识。别小看了这些基础，我朋友水平挺可以的，结果就栽在一道基础知识的问题上，和高薪无缘。

希望这些知识点对大家有所帮助，愿大家从达内出来后都能找到高薪的工作！

第一，谈谈 final, finally, finalize 的区别。 final 用于声明属性，方法和类，分别表示属性不可变，方法不可覆盖，类不可继承。finally 是异常处理语句结构的一部分，表示总是执行。finalize 是 Object 类的一个方法，在垃圾收集器执行的时候会调用被回收对象的此方法，可以覆盖此方法提供垃圾收集时的其他资源回收，例如关闭文件等。

第二，Anonymous Inner Class (匿名内部类) 是否可以 extends(继承) 其它类，是否可以 implements(实现) interface(接口)? 可以继承其他类或完成其他接口，在 swing 编程中常用此方式。

第三，Static Nested Class 和 Inner Class 的不同，说得越多越好(面试题有的很笼统)。 Static Nested Class 是被声明为静态 (static) 的内部类，它可以不依赖于外部类实例被实例化。而通常的内部类需要在外部类实例化后才能实例化。

第四，&和&&的区别。 &是位运算符，表示按位与运算，&&是逻辑运算符，表示逻辑与 (and)。

第五，HashMap 和 Hashtable 的区别。 HashMap 是 Hashtable 的轻量级实现 (非线程安全的实现)，他们都完成了 Map 接口，主要区别在于 HashMap 允许空 (null) 键值 (key)，由于非线程安全，效率上可能高于 Hashtable。

第六，Collection 和 Collections 的区别。 Collection 是集合类的上级接口，继承与他的接口主要有 Set 和 List。Collections 是针对集合类的一个帮助类，他提供一系列静态方法实现对各种集合的搜索、排序、线程安全化等操作。

第七，什么时候用 assert。 1.4 新增关键字 (语法)，用于测试 boolean 表达式状态，可用于调试程序。使用方法 assert <boolean 表达式>，表示如果表达式为真 (true)，则下面的语句执行，否则抛出 AssertionError。另外的使用方

式 `assert < boolean 表达式>:<other 表达式>`, 表示如果表达式为真, 后面的表达式忽略, 否则后面表达式的值用于 `AssertionError` 的构建参数。注意编译时要增加 `-source 1.4` 参数, 否则报错。]运行时要增加 `-ea` 参数, 否则 `assert` 行被忽略

第八, GC 是什么? 为什么要有 GC? GC 是垃圾收集的意思 (Garbage Collection), 内存处理是编程人员容易出现问题的地方, 忘记或者错误的内存回收会导致程序或系统的不稳定甚至崩溃, Java 提供的 GC 功能可以自动监测对象是否超过作用域从而达到自动回收内存的目的, Java 语言没有提供释放已分配内存的显示操作方法。

第九, `String s = new String("xyz");` 创建了几个 String Object? 两个

第十, `Math.round(11.5)` 等於多少? `Math.round(-11.5)` 等於多少?
`Math.round(11.5)==12`
`Math.round(-11.5)==-11`
`round` 方法返回与参数最接近的长整数, 参数加 $1/2$ 后求其 `floor`.

第十一, `short s1 = 1; s1 = s1 + 1;` 有什么错? `short s1 = 1; s1 = 1;` 有什么错? `short s1 = 1; s1 = s1 - 1;` (`s1 - 1` 运算结果是 `int` 型, 需要强制转换类型) `short s1 = 1; s1 = 1;` (可以正确编译)

第十二, `sleep()` 和 `wait()` 有什么区别? `sleep` 是线程类 (`Thread`) 的方法, 导致此线程暂停执行指定时间, 给执行机会给其他线程, 但是监控状态依然保持, 到时后会自动恢复。调用 `sleep` 不会释放对象锁。`wait` 是 `Object` 类的方法, 对此对象调用 `wait` 方法导致本线程放弃对象锁, 进入等待此对象的等待锁定池, 只有针对此对象发出 `notify` 方法 或 `notifyAll()` 后本线程才进入对象锁定池准备获得对象锁进入运行状态。

第十三, Java 有没有 `goto`? 没有。很十三的问题, 如果哪个面试的问到这个问题, 我劝你还是别进这家公司。(完) (job.sohu.com)

第十四, 数组有没有 `length()` 这个方法? `String` 有没有 `length()` 这个方法? 数组没有 `length()` 这个方法, 有 `length` 的属性。`String` 有 `length()` 这个方法。

第十五, `Overload` 和 `Override` 的区别。`Overloaded` 的方法是否可以改变返回值的类型? 方法的重写 `Overriding` 和重载 `Overloading` 是 Java 多态性的不同表现。重写 `Overriding` 是父类与子类之间多态性的一种表现, 重载 `Overloading` 是一个类中多态性的一种表现。如果在子类中定义某方法与其父类有相同的名称和参数, 我们说该方法被重写 (`Overriding`)。子类的对象使用这个方法时, 将调用子类中的定义, 对它而言, 父类中的定义如同被“屏蔽”了。如果在一个类中定义了多个同名的方法, 它们或有不同的参数个数或有不同的参数类型, 则称为方法的重载 (`Overloading`)。重载的方法是可以改变返回值的类型。

第十六, `Set` 里的元素是不能重复的, 那么用什么方法来区分重复与否呢? 是用 `==` 还是 `equals()`? 它们有何区别? `Set` 里的元素是不能重复的, 那么用 `iterator()`

方法来区分重复与否。equals() 是判读两个 Set 是否相等。equals() 和 == 方法决定引用值是否指向同一对象 equals() 在类中被覆盖，为的是当两个分离的对象的内容和类型相配的话，返回真值。

第十七，给我一个你最常见到的 runtime exception。ArithmeticException, ArrayStoreException, BufferOverflowException, BufferUnderflowException, CannotRedoException, CannotUndoException, ClassCastException, CMMException, ConcurrentModificationException, DOMException, EmptyStackException, IllegalArgumentException, IllegalMonitorStateException, IllegalPathStateException, IllegalStateException, ImagingOpException, IndexOutOfBoundsException, MissingResourceException, NegativeArraySizeException, NoSuchElementException, NullPointerException, ProfileDataException, ProviderException, RasterFormatException, SecurityException, SystemException, UndeclaredThrowableException, UnmodifiableSetException, UnsupportedOperationException

第十八，error 和 exception 有什么区别？error 表示恢复不是不可能但很困难的情况下的一种严重问题。比如说内存溢出。不可能指望程序能处理这样的情况。exception 表示一种设计或实现问题。也就是说，它表示如果程序运行正常，从不会发生的情况。

第十九，List, Set, Map 是否继承自 Collection 接口？List, Set 是 Map 不是

第二十，abstract class 和 interface 有什么区别？声明方法的存在而不去实现它的类被叫做抽象类（abstract class），它用于提供一个体现某些基本行为的类，并为该类声明方法，但不可以在该类中实现该方法的情况。不能创建 abstract 类的实例。然而可以创建一个变量，其类型是一个抽象类，并让它指向具体子类的一个实例。不能有抽象构造函数或抽象静态方法。Abstract 类的子类为它们父类中的所有抽象方法提供实现，否则它们也是抽象类为。取而代之，在子类中实现该方法。知道其行为的其它类可以在类中实现这些方法。接口（interface）是抽象类的变体。在接口中，所有方法都是抽象的。多继承性可通过实现这样的接口而获得。接口中的所有方法都是抽象的，没有一个有程序体。接口只可以定义 static final 成员变量。接口的实现与子类相似，除了该实现类不能从接口定义中继承行为。当类实现特殊接口时，它定义（即将程序体给予）所有这种接口的方法。然后，它可以在实现了该接口的类的任何对象上调用接口的方法。由于有抽象类，它允许使用接口名作为引用变量的类型。通常的动态联编将生效。引用可以转换到接口类型或从接口类型转换，instanceof 运算符可以用来决定某对象的类是否实现了接口。

第二十一，abstract 的 method 是否可同时是 static，是否可同时是 native，是否可同时是 synchronized？都不能

第二十二，接口是否可继承接口？抽象类是否可实现（implements）接口？抽象类是否可继承实体类（concrete class）？接口可以继承接口。抽象类可以实现

(implements)接口, 抽象类是否可继承实体类, 但前提是实体类必须有明确的构造函数。

第二十三, 启动一个线程是用 run() 还是 start()? 启动一个线程是调用 start() 方法, 使线程所代表的虚拟处理机处于可运行状态, 这意味着它可以由 JVM 调度并执行。这并不意味着线程就会立即运行。run() 方法可以产生必须退出的标志来停止一个线程。

第二十四, 构造器 Constructor 是否可被 override? 构造器 Constructor 不能被继承, 因此不能重写 Overriding, 但可以被重载 Overloading。

第二十五, 是否可以继承 String 类? String 类是 final 类故不可以继承。

第二十六, 当一个线程进入一个对象的一个 synchronized 方法后, 其它线程是否可进入此对象的其它方法? 不能, 一个对象的一个 synchronized 方法只能由一个线程访问。

第二十七, try {} 里有一个 return 语句, 那么紧跟在这个 try 后的 finally {} 里的 code 会不会被执行, 什么时候被执行, 在 return 前还是后? 会执行, 在 return 前执行。

第二十八, 编程题: 用最有效率的方法算出 2 乘以 8 等於几? 有 C 背景的程序员特别喜欢问这种问题。2 << 3

第二十九, 两个对象值相同 (x.equals(y) == true) 但却可能有不同的 hash code, 这句话对不对? 不对, 有相同的 hash code。

第三十, 当一个对象被当作参数传递到一个方法后, 此方法可改变这个对象的属性, 并可返回变化后的结果, 那么这里到底是值传递还是引用传递? 是值传递。Java 编程语言只由值传递参数。当一个对象实例作为一个参数被传递到方法中时, 参数的值就是对该对象的引用。对象的内容可以在被调用的方法中改变, 但对对象的引用是永远不会改变的。

第三十一, switch 是否能作用在 byte 上, 是否能作用在 long 上, 是否能作用在 String 上? switch(expr1) 中, expr1 是一个整数表达式。因此传递给 switch 和 case 语句的参数应该是 int、short、char 或者 byte。long, string 都不能作用于 switch。

第三十二, 编程题: 写一个 Singleton 出来。Singleton 模式主要作用是保证在 Java 应用程序中, 一个类 Class 只有一个实例存在。一般 Singleton 模式通常有几种形式: 第一种形式: 定义一个类, 它的构造函数为 private 的, 它有一个 static 的 private 的该类变量, 在类初始化时实例化, 通过一个 public 的 getInstance 方法获取对它的引用, 继而调用其中的方法。public class Singleton { private Singleton() {} //在自己内部定义自己一个实例, 是不是很奇怪? //注意这是 private 只供内部调用 private static

Singleton instance = new Singleton(); //这里提供了一个供外部访问本 class 的静态方法, 可以直接访问 public static Singleton getInstance() { return instance; } } 第二种形式: public class Singleton { private static Singleton instance = null; public static synchronized Singleton getInstance() { //这个方法比上面有所改进, 不用每次都进行生成对象, 只是第一次 //使用时生成实例, 提高了效率! if (instance==null) instance=new Singleton();return instance; } } 其他形式: 定义一个类, 它的构造函数为 private 的, 所有方法为 static 的。一般认为第一种形式要更加安全些

神州数码面试集合

问答题:

◆写一段 Jdbc 连 Oracle 的程序. 注意步骤什么地方抛出什么异常, 最后关闭资源

◆EJB 规范规定 EJB 中禁止的操作有哪些? (15 分)

共有 8 点, 答出下列 3-4 点得满分.

1. 不能操作线程和线程 API (线程 API 指非线程对象的方法如 notify, wait 等)
2. 不能操作 awt
3. 不能实现服务器功能
4. 不能对静态属性读取.
5. 不能使用 IO 操作直接存取文件系统.
6. 不能加载本地库.
7. 不能将 this 作为变量和返回.
8. 不能循环调用.

◆简述逻辑操作 (&, |, ^) 与条件操作 (&&, ||) 的区别. (15 分)

区别主要答两点:

a. 条件操作只能操作布尔型的, 而逻辑操作不仅可以操作布尔型, 而且可以操作数值型

b. 逻辑操作不会产生短路. 如:

```
int a = 0;
int b = 0;
if( (a = 3) > 0 || (b = 3) > 0 ) //操后 a =3, b=0.
if( (a = 3) > 0 & (b = 3) > 0 ) //操后 a =3, b=3.
```

答对第一点得 5 分, 答对第二点得 10 分.

◆下面程序运行会发生什么结果?如果有错误, 如何改正? (15 分)

```
interface A{
    int x = 0;
```

```

    }
class B{
    int x =1;
}
class C
    extends B implements A {
        public void pX() {
            System.out.println(x);
        }
        public static void main(String[] args) {
            new C().pX();
        }
    }
}

```

本题在编译时会发生错误(错误描述不同的 JVM 有不同的信息, 意思就是未明确的 x 调用,

两个 x 都匹配, 就象在同时 import java.util 和 java.sql 两个包时直接声明 Date 一样)

本题主要考察对接口和类的最最基本的结构的了解. 对于父类的变量, 可以用 super.x 来

明确, 而接口的属性默认隐含为 public static final. 所以可以通过 A.x 来明确.

◆ 简述 Java Server Page 和 Servlet 的联系和区别。(20 分)

jsp 和 servlet 的实质是一样的, JSP 在处理客户端请求前会被编译成 servlet(对于不同服务器中间件被编译后的 service 文件存放的位置不同, tomcat 在

\$catalina-home/work/catalina/localhost/app-name/org/apache/jsp 目录下), JSP declaration 中定义的变量和立法在运行时成为对应 servlet 的成员变量和方法; scripting 会成为 _jspService 方法的语句。JSP 的生命周期受到 _jspInit()、_jspService()、_jspDestroy() 的控制, 多数情况下 Jsp 适合前台页面显示, 而 Servlet 用于控制页面逻辑或后台处理

本题不用多说, 在答相同点时应该明确知道 jsp 编译后是“类 servlet”而“不是 Servlet”,

答区别时应该回答出“侧重于(视图/控制逻辑)”. 其它可根据情况加减分值. 知识很简单,

但从面试的角度看, 被试者不仅要能知道它们的区别, 而且要能比较准确地表达出来(以后写文档要能让别人看得懂, 不产生歧义), 回答“jsp 编译后就是 servlet”视为错误, 回答“jsp 用于视图, servlet 用于控制逻辑”视为错误, 应该用侧重于, 主要(多数)用于等词语表达.

◆ XML 文档定义有几种形式? 它们之间有何本质区别?

解析 XML 文档有哪几种方式? (20 分)

本题三个答题点:

a: 两种形式 dtd, schema

b: 本质区别:schema 本身是 xml 的, 不像 DTD 有其特有的语法。

主要区别:

一.XML Schema 是 XML 文档, 不像 DTD 有其特有的语法。对开发人员来说, 你不需要懂两种语法就能编写一个格式良好的 XML 文档。对开发 XML 分析器来说, XML Schema 由于也是 XML 语法, 实现及支持起来就更方便了。同时 XML Schema 继承了 XML 可扩展的优点。

二.定义数据类型。在一个 DTD 文件中只能把数据声明为字符串类型或字符串类型的子元素, 如 PCDATA、CDATA、ID 等。而在 XML Schema 中, 可以定义跟编程语言同样丰富的数据类型, 如整型, 浮点型, 布尔型, 日期型等。这样的好处是显而易见的, 在你编写程序使用一个整型数据时, 如用 DTD 定义的, 就必须将它从字符型转化为整型, 而 XML Schema 则可以直接定义为整型。

c: 两种主要方式: dom, sax. 答出两种得全分, 如能答出 saxt 或其它 (在答出 dom, sax 的基础上, 如果应试者认为其它方式也可以视为对 xml 的解析应该允许, 但没有答出 dom, sax 把其它方式说成是对 XML 的解析不得分) 应该加分。

◆简述 synchronized 和 java.util.concurrent.locks.Lock 的异同 ? (15 分)

主要相同点:

Lock 能完成 synchronized 所实现的所有功能. (其它不重要)

主要不同点:

Lock 有比 synchronized 更精确的线程语义和更好的性能 (在相同点中回答此点也行)

synchronized 会自动释放锁. 而 Lock 一定要求程序员手工释放. 并且必须在 finally 从句中释放, 如果没有答出在 finally 中释放不得分. 就如 Connection 没有在 finally 中关闭一样. 连最基本的资源释放都做不好, 还谈什么多线程编程.

◆ 什么是聚集索引, 什么是非聚集索引, 什么又是主键?

表中经常有一个列或列的组合, 其值能唯一地标识表中的每一行。这样的一列或多列称为表的主键。

聚集索引确定表中数据的物理顺序。聚集索引类似于电话簿, 后者按姓氏排列数据。由于聚集索引规定数据在表中的物理存储顺序,

因此一个表只能包含一个聚集索引。但该索引可以包含多个列 (组合索引), 就像电话簿按姓氏和名字进行组织一样。

非聚集索引与课本中的索引类似。数据存储在一个地方, 索引存储在另一个地方, 索引带有指针指向数据的存储位置。

索引中的项目按索引键值的顺序存储, 而表中的信息按另一种顺序存储 (这可以由聚集索引规定)。

如果在表中未创建聚集索引，则无法保证这些行具有任何特定的顺序。

◆软件开发过程，遵循什么规范

软件开发过程大的方面来讲包括需求分析、软件设计和实现(概要设计、详细设计、编码)、软件测试(单元测试、代码检查、功能测试、系统测试、验收测试)和交付

规范：1 命名规范 2 代码规范 3 工程文件组织规范 4 类组织规范 5 用户界面规范 6 疑难解答和 Bug 调试方法

◆作用域 public, protected, private, 以及不写时的区别

public 可以被本身和其他的包中的类引用，protected 可以被同一个包内的类以及不同包中的子类引用，private 只有自己的类中可以使用。不写的时候只能被同一个包中的类引用。

◆char 型变量能不能定义为一个中文?为什么?

可以定义。因为 java 中的 char 是两个字节，可以存储中文。

◆jsp 有哪些内置对象?作用分别是什么?

JSP 共有以下 9 种基本内置组件(可与 ASP 的 6 种内部组件相对应)：

request 用户端请求，此请求会包含来自 GET/POST 请求的参数

response 网页传回用户端的回应

pageContext 网页的属性是在这里管理

session 与请求有关的会话期

application servlet 正在执行的内容

out 用来传送回应的输出

config servlet 的构架部件

page JSP 网页本身

exception 针对错误网页，未捕捉的例外

◆jsp 有哪些动作?作用分别是什么?

JSP 共有以下 6 种基本动作

jsp:include: 在页面被请求的时候引入一个文件。

jsp:useBean: 寻找或者实例化一个 JavaBean。

jsp:setProperty: 设置 JavaBean 的属性。

jsp:getProperty: 输出某个 JavaBean 的属性。

jsp:forward: 把请求转到一个新的页面。

jsp:plugin: 根据浏览器类型为 Java 插件生成 OBJECT 或 EMBED 标记

◆ ◆JSP 中 INCLUDE 两种实现方式的区别?

动态、静态两种。

动态 INCLUDE 用 jsp:include 动作实现

<jsp:include page="included.jsp" flush="true" />它总是会检查所含文件中的变化，适合用于包含动态页面，并且可以带参数

静态 INCLUDE 用 include 伪码实现，定不会检查所含文件的变化，适用于包含静态页面 <%@ include file="included.htm" %>

◆JSP 中两种跳转方式是什么? 区别是什么?

<jsp:include page="included.jsp" flush="true">

```
<jsp:forward page= "nextpage.jsp"/>
```

前者页面不会转向 include 所指的页面，只是显示该页的结果，主页面还是原来的页面。执行完后还会回来，相当于函数调用。并且可以带参数。后者完全转向新页面，不会再回来。相当于 go to 语句。

◆JSP 有哪些中文问题，如何解决？

1JSP 的 charset

2JSP SERVER 编译 JSP 是使用的字符集，默认和操作系统是一样的，可能需要进行修改

3 你的 BEAN 编译时候所采用的字符集，一般用 JavaC 编译，都是和操作系统一样的，但你可以通过带参数来改变。

4 和数据库连接时，大多数 JDBC 驱动程序都可以设置其字符集的，这样 JDBC 驱动会自己进行转换，也不会出现汉字问题。

◆说一说 Servlet 的生命周期？

servlet 有良好的生存期的定义，包括加载和实例化、初始化、处理请求以及服务结束。这个生存期由 javax.servlet.Servlet 接口的 init, service 和 destroy 方法表达。

Servlet 的生命周期是当服务器装载运行 servlets，接收来自客户端的多个请求并且返回数据给客户端，然后再删除移开 servlets 的时间。

◆Class.forName 的作用？为什么要用？

Class.forName 是寻找并加载一个类。用它是为了 load 你指定名称的 class。

◆MVC 的各个部分都有那些技术来实现？如何实现？

MVC 是 Model—View—Controller 的缩写。“Model”代表的是应用的业务逻辑(通过 JavaBean, EJB 组件实现)，“View”是应用的表示面(由 JSP 页面产生)，“Controller”是提供应用的处理过程控制(一般是一个 Servlet)，通过这种设计模型把应用逻辑，处理过程和显示逻辑分成不同的组件实现。这些组件可以进行交互和重用。

◆javascript 如何校验数字型？

isNaN(parseFloat(xxx)) 如果为 true，则不是数字。

```
var v = document.all.Telephone.value;
if(isNaN(v)){
alert("全是数字");
}
```

◆html 样式表作用，原理。

大量减少网页代码，原理就是在网页中自定义样式表的选择符，然后在网页中大量引用这些选择符。目前大部分网站都是使用 class 来引用的。

◆ 画出 tcp/ip 协议分层，解释 arp 协议的作用

应用层 (Application)	HTTP、Telnet、FTP、SMTP、SNMP
----------------------	---------------------------

传 输 层 (Transport)	TCP、UDP
网 间 网层 (Internet)	IP 【ARP、RARP、ICMP】
网络接口层 (Network)	Ethernet、X.25、SLIP、PPP

在以太网(Ethernet)中,一个网络设备要和另一个网络设备进行直接通信,除了知道目标设备的网络层逻辑地址(如 IP 地址)外,还要知道目标设备的第二层物理地址(MAC 地址)。ARP 协议的基本功能就是通过目标设备的 IP 地址,查询目标设备的 MAC 地址,以保证通信的顺利进行。

当一个网络设备需要和另一个网络设备通信时,它首先把目标设备的 IP 地址与自己的子网掩码进行“与”操作,以判断目标设备与自己是否位于同一网段内。如果目标设备在同一网段内,并且源设备没有获得与目标 IP 地址相对应的 MAC 地址信息,则源设备以第二层广播的形式(目标 MAC 地址为全 1)发送 ARP 请求报文,在 ARP 请求报文中包含了源设备与目标设备的 IP 地址。同一网段中的所有其他设备都可以收到并分析这个 ARP 请求报文,如果某设备发现报文中的目标 IP 地址与自己的 IP 地址相同,则它向源设备发回 ARP 响应报文,通过该报文使源设备获得目标设备的 MAC 地址信息。

如果目标设备与源设备不在同一网段,则源设备首先把 IP 分组发向自己的缺省网关(Default Gateway),由缺省网关对该分组进行转发。如果源设备没有关于缺省网关的 MAC 信息,则它同样通过 ARP 协议获取缺省网关的 MAC 地址信息。

为了减少广播量,网络设备通过 ARP 表在缓存中保存 IP 与 MAC 地址的映射信息。在一次 ARP 的请求与响应过程中,通信双方都把对方的 MAC 地址与 IP 地址的对应关系保存在各自的 ARP 表中,并在后续的通信中使用。ARP 表使用老化机制,删除在一段时间内没有使用过的 IP 与 MAC 地址的映射关系。

◆ 进程的那些状态（就绪，运行，阻塞，挂起）的转换图 以及转换的条件

编程题

◆找出两个字符串中最大子字符串,如“abractyeyt”,“dgdsaeactyey”的最大子串为“actyet”

```
int main(int argc, char* argv[])
{
    char str1[] = "abractyeyt", str2[] = "dgdsaeactyey";
    char *strResult = MaxSubString(str1, str2);
    printf("str1=%s\nstr2=%s\nMaxSubString=%s\n", str1, str2,
strResult);
}
```

```

    return getchar();
}

```

◆删除字符串中的数字并压缩字符串，如字符串”abc123de4fg56”处理后变为”abcdefg”。注意空间和效率。（下面的算法只需要一次遍历，不需要开辟新空间，时间复杂度为 $O(N)$ ）

```

#include "stdafx.h"
void delNum(char *str)
{
    int i, j=0;
    for(i=j=0; str[i] && (str[i]<'0' || str[i]>'9'); j=++i); // 找到串
    中第一个数字的位子
    for(; str[i]; i++) // 从串中第一个数字的位置开始，逐个放入后
    面的非数字字符
        if(str[i]<'0' || str[i]>'9')
            str[j++] = str[i];
    str[j] = '\0';
}
int main(int argc, char* argv[])
{
    char str[] = "abc123ef4g4h5";
    printf("%s\n", str);
    delNum(str);
    printf("%s\n", str);
    return getchar();
}

```

◆ 有一百个整数，其中有负数，找出连续三个数之和最大的部分。

C++ 部分

◆把字符串转化成整型，不能用系统函数

```

#include <stdio.h>

int _atoi(char *str)
{
    static int i;
    sscanf(str, "%d", &i);
    return i;
}

int main()
{
    printf("%d\n", _atoi("654321"));
}

```

```
printf("%d\n",_atoi("12345abc"));
return 0;
}
```

推理题：

- ◆24 个人，要求 5 个人一排，排 6 排如何排？
排成六边形
- ◆8 个人跑步比赛结果，给出系列条件：那个排名在那个前面、有一个第 4 名……，让你推测出实际的名次。（略）
- ◆两根蜡烛，每根单独燃烧一个小时燃尽。如何通过两根蜡烛的燃烧判断 45 分钟？
- ◆请估计北京有多少加油站？并简述理由。
- ◆北京有 200 万量机动车，加油站工作时间按每天 18 小时，假设每个油站能同时工作 6 个油泵，每次为车加油时间为 3 分钟，每天一个加油站工作车辆数： $18*60*6/3=2160$ 辆，所需加油站数量= $2000000/2160=926$ 个加油站。
- ◆给你一个非常困难的问题，你将如何解决它。
询问有解决能力的人该如何解决并在以后的工作之中将它也变成自己可以解决的问题

Tarena Wuhan

长城软件笔试

感谢 WHSD/ASD0701 班孟立松同学提供
Oracle 笔试考题

- 一、请简述范式中第二范式，第三范式，BCNF 的区别
- 二、请按照要求写出 sql 语句

表内容:

2005-05-09 胜
2005-05-09 胜
2005-05-09 负
2005-05-09 负
2005-05-10 胜
2005-05-10 负
2005-05-10 负

如果要生成下列结果, 怎样写 sql?

	胜	负
2005-05-09	2	2
2005-05-10	1	2

```
create table #tmp(rq varchar(10), shangfu nchar(1))
```

```
insert into #tmp values('2005-05-09','胜')
insert into #tmp values('2005-05-09','胜')
insert into #tmp values('2005-05-09','负')
insert into #tmp values('2005-05-09','负')
insert into #tmp values('2005-05-10','胜')
insert into #tmp values('2005-05-10','负')
insert into #tmp values('2005-05-10','负')
```

三、按要求写出 sql 语句

表中有 ABC 三列, 用 sql 语句实现: 当 A 列大于 B 列时选择 A 列, 否则选择 B 列; 当 B 列大于 C 列时选择 B 列, 否则选择 C 列。

四、按要求写一个日期判断的 sql 语句

请取出 tb_sent 表中日期 (SentTime 字段) 为当天的所有记录 (SentTime 字段为 datetime 型, 包含日期与时间)

五、有一张表, 里面有三个字段: 语文、数学、英语。其中有三条记录分别表示语文 70 分, 数学 80 分, 英语 58 分, 请用一条 sql 语句查询出这三条记录并按照以下条件显示出来 (并写出思路)

大于等于 80 分表示优秀, 大于等于 60 分表示及格, 小于 60 分表示不及格。

显示格式:

语文	数学	英语
及格	优秀	不及格

六、请用一个 sql 语句得出结果

从 table1, table2 中取出如 table3 格式的数据

table1

月份 mon	部门 dep	业绩 yj
--------	--------	-------

一月份	01	10
一月份	02	10
一月份	03	5
二月份	02	8
二月份	04	9
三月份	03	8

table2

部门 dep	部门名称 dname
01	国际业务一部
02	国际业务二部
03	国际业务三部
04	国际业务部

table3(result)

部门 dep	一月份	二月份	三月份
01	10	null	null
02	10	8	null
03	null	5	8
04	null	null	9

七、用一个 sql 语句完成下列查询

已知表 学生 student 及其属性 sid (学号) sname (姓名)

课程 course 及其属性 cid (课程号) cname (课程名称)

选修记录 sc 及其属性 sid (学号) cid (课程号) scscore (选修成绩)

- 求
- 1) 至少选修了学生 4 号 (学号) 1, 2, 4, 5 选修的全部课程的学生号码及姓名
 - 2) 选修成绩在平均分以下超过平均的全部学生及全部课程成绩

部分 java 题目

1. 描述下 jvm 加载 class 文件的原理机制。
2. 请简述 servlet 在 struts 中的地址，一般用来做什么工作。
3. servlet 在 j2ee 架构中一般用来做什么工作。
4. 在 servlet 中如何处理 request 和 response 的中文编码问题。
5. 请用 java 语言描述一个二叉树前序遍历的过程。

EDS 的面试题

- 1、Servlet 的生命周期有哪些方法？
- 2、Servlet 和 JSP 有什么区别？
- 3、描述一下用 JDBC 连接一个数据库？
- 4、Statement 和 PreparedStatement 的区别？
- 5、Hibernate 中实现继承有几种解决方法？
- 6、用 Java 连接数据库有几种解决方法？

- 7、EJB 有几种 Bean?
- 8、Stateful、Stateless Session Bean 的区别?
- 9、怎么配置 Spring 相关的属性?
- 10、用过 ThreadLocal 了么?

Tarena Wuhan

盛意科技 java 面试

感谢 Vince 提供, Eric 给出参考答案

三、Java 部分

1、有个 List 中存储了 Person 类型的对象，请用代码删除 List 中 Person 对象的 kind 属性大于“A”或“a”的所有对象

答:

```
public void remove(List persons){
    for(int i=0;i<persons.size();i++){
        Person p = (Person)persons.get(i);
        String str = p.getkind();
        if("A".compareTo(str)<0||"a".compareTo(str)<0){
            System.out.println(str);
            i--;
            persons.remove(p);
        }
    }
}
```

2、请写出下面程序 1 到 5 的执行顺序和程序输出结果:

```
class Singleton{
    1) private static Singleton obj = new Singleton();
    2) public static int counter1 = 5;
    3) public static int counter2 = 8;
    4) private Singleton(){
        counter1++;
        counter2++;
    }
    5) public static Singleton getInstance()
        return obj;
}
}
public class MyMain {
    public static void main(String[] args) {
        Singleton obj = Singleton.getInstance();
        System.out.println("obj.counter1 = "+obj.counter1);
        System.out.println("obj.counter2 = "+obj.counter2);
    }
}
```

答: 执行顺序 1) ----4) ----2) ----3) ---5)

输出结果: obj.counter1 = 1
obj.counter2 = 1

3、请解释什么是 MVC 模式，请用一种你熟悉的框架解释他们是怎么实现 MVC 模式的。

比如: Struts

MVC 通过将问题分为三个类别来帮助解决单一模块方法所遇到的某些问题:

- **Model (模型)**

模型包含应用程序的核心功能。模型封装了应用程序的状态。有时它包含的唯一功能就是状态。它对视图或控制器一无所知。

- **View (视图)**

视图提供模型的表示。它是应用程序的外观。视图可以访问模型的读方法，但不能访问写方法。此外，它对控制器一无所知。当更改模型时，视图应得到通知。

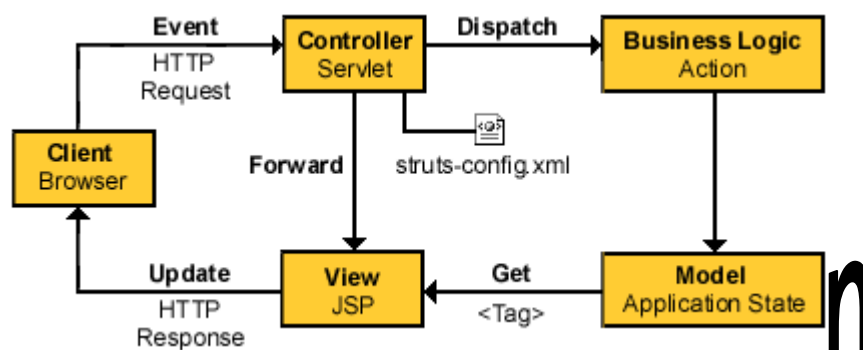
- **Controller (控制器)**

控制器对用户的输入作出反应。它创建并设置模型。

Struts, MVC 2 的一种实现

Struts 是一组相互协作的类、servlet 和 JSP 标记，它们组成一个可重用的 MVC 2 设计。这个定义表示 Struts 是一个框架，而不是一个库，但 Struts 也包含了丰富的标记库和独立于该框架工作的实用程序类。图 5 显示了 Struts 的一个概览。

图 5. Struts 概览



Struts 概览

- **Client browser (客户浏览器)**
来自客户浏览器的每个 HTTP 请求创建一个事件。Web 容器将用一个 HTTP 响应作出响应。
- **Controller (控制器)**
控制器接收来自浏览器的请求，并决定将这个请求发往何处。就 Struts 而言，控制器是以 servlet 实现的一个命令设计模式。struts-config.xml 文件配置控制器。
- **业务逻辑**
业务逻辑更新模型的状态，并帮助控制应用程序的流程。就 Struts 而言，这是通过作为实际业务逻辑“瘦”包装的 Action 类完成的。Model (模型) 的状态
模型表示应用程序的状态。业务对象更新应用程序的状态。ActionForm bean 在会话级或请求级表示模型的状态，而不是在持久级。JSP 文件使用 JSP 标记读取来自 ActionForm bean 的信息。
- **View (视图)**
视图就是一个 JSP 文件。其中没有流程逻辑，没有业务逻辑，也没有模型信息 -- 只有标记。标记是使 Struts 有别于其他框架（如 Velocity）的因素之一。

4、说出 ArrayList, Vector, LinkedList 的存储性能和特性

答: ArrayList 和 Vector 都是使用数组方式存储数据, 此数组元素数大于实际存储的数据以便增加和插入元素, 它们都允许直接按序号索引元素, 但是插入元素要涉及数组元素移动等内存操作, 所以索引数据快而插入数据慢, Vector 由于使用了 synchronized 方法(线程安全), 通常性能上较 ArrayList 差, 而 LinkedList 使用双向链表实现存储, 按序号索引数据需要进行前向或后向遍历, 但是插入数据时只需要记录本项的前后项即可, 所以插入速度较快。

5、分别列出 List, Set, Map 的两个实现类, 并说明这两个实现类的不同之处

答:

List: ArrayList 与 Vector

一.同步性:Vector 是线程安全的, 也就是说是同步的, 而 ArrayList 是线程程序不安全的, 不是同步的

二.数据增长:当需要增长时,Vector 默认增长为原来一倍, 而 ArrayList 却是原来的一半

Set: HashSet 与 TreeSet

一. HashSet 实现 Set 接口, 由哈希表(实际上是一个 HashMap 实例)支持。它不保证集合的迭代顺序; 特别是它不保证该顺序恒久不变。此类允许使用 null 元素。

二. TreeSet 实现 Set 接口, 该接口由 TreeMap 实例支持。此类保证排序后的 set 按照升序排列元素, 根据使用的构造方法不同, 可能会按照元素的自然顺序 进行排序, 或按照在创建 set 时所提供的比较器进行排序。

Map: HashMap 与 Hashtable。

一.历史原因:Hashtable 是基于陈旧的 Dictionary 类的, HashMap 是 Java 1.2 引进的 Map 接口的一个实现

二.同步性:Hashtable 是线程安全的, 也就是说是同步的, 而 HashMap 是线程程序不安全的, 不是同步的, 由于非线程安全, 效率上可能高于 Hashtable。

三.值: 只有 HashMap 可以让你将 null 作为一个表的条目的 key 或 value

6、Overload 和 Override 的区别。Overloaded 的方法是否可以改变返回值的类型

答: 方法的重写 Overriding 和重载 Overloading 是 Java 多态性的不同表现。重写 Overriding 是父类与子类之间多态性的一种表现, 重载 Overloading 是一个类中多态性的一种表现。如果在子类中定义某方法与其父类有相同的名称和参数, 我们说该方法被重写 (Overriding)。子类的对象使用这个方法时, 将调用子类中的定义, 对它而言, 父类中的定义如同被"屏蔽"了。如果在一个类中定义了多个同名的方法, 它们或有不同的参数个数或有不同的参数类型, 则称为方法的重载(Overloading)。Overloaded 的方法是可以改变返回值的类型

7、JSP 的内置对象及方法。

答:

1).request 表示 HttpServletRequest 对象。它包含了有关浏览器请求的信息, 并且提供了几个用于获取 cookie, header, 和 session 数据的有用的方法。

2).response 表示 HttpServletResponse 对象, 并提供了几个用于设置送回 浏览器的响应的方法 (如 cookies, 头信息等)

3). out 对象是 javax.jsp.JspWriter 的一个实例, 并提供了几个方法使你能用于向浏览器回送输出结果。

4). pageContext 表示一个 javax.servlet.jsp.PageContext 对象。它是用于方便存取各种范围的

名字空间、servlet 相关的对象的 API，并且包装了通用的 servlet 相关功能的方法。

- 5).session 表示一个请求的 javax.servlet.http.HttpSession 对象。Session 可以存贮用户的状态信息
- 6).application 表示一个 javax.serve.ServletContext 对象。这有助于查找有关 servlet 引擎和 servlet 环境的信息
- 7). config 表示一个 javax.servlet.ServletConfig 对象。该对象用于存取 servlet 实例的初始化参数。
- 8).page 表示从该页面产生的一个 servlet 实例
- 9).exception 针对错误网页，未捕捉的例外

Tarena Wuhan